

DTU



Lecture 1: Introduction

Mathematical Programming Modeling

Welcome to the course

This course will teach you how to formulate optimization models and how to solve them using optimization software.

Lecture 1: Outline

- Course introduction
- Formulating our first model
- Julia/JuMP for solving mathematical optimization models
- Modeling introduction
- Basic LP model in JuMP

Who are we ?

- Lecturers:
 - Associate Professor Thomas Stidsen, email: thst@dtu.dk
 - Professor Stefan Røpke, email: ropke@dtu.dk
 - Mikkel Lassen Johansen, email: mlajo@dtu.dk
- Teaching Assistants:
 - Mikkel Lassen Johansen, email: mlajo@dtu.dk
 - Kristine Børsting, email: kribo@dtu.dk
 - Agita Solzemniece, email: agiso@dtu.dk
 - Daniel Guerrero Domínguez: dgudo@dtu.dk
- Language: English (or Danish to Mikkel, Kristine, Stefan or Thomas).

General Course Info

- Period: 5/1 - 23/1
- Time: 9.00-17.00
- Accounts for 5 ECTS points
- Evaluation:
 - 7 point scale, based on written exam 23/1, 2025, 3 hours, in two parts:
 - Multiple choice (on math. prog. modeling)
 - Create a model and then implement in julia/JuMP program

Course plan

Chapter 1 in the course book (P. 6), contains a (detailed) course plan. This plan specifies:

- Week 1:
 - Linear Programming
 - Simple Mixed Integer Programming
- Week 2:
 - More complex Mixed Integer Programming assignments
- Week 3:
 - Math. heuristics
 - Multi-Objective Optimization
 - Exam-practice day
 - Exam

The Exam

- Takes place on Friday January 23/1 2026 (time not decided yet)
- There is a trial exam, Wednesday Wednesday 21/1, 9.30-12.30, in exercise room 069 and 070, building 358. Introduction by Thomas, 9.00-9.30
- Exam is an *offline* exam, no use of AI !

Course material and groups

- **Course book**, download pdf from DTU-Learn (we may update the version during the course)
- **Assignment Answers**, answers available every day 17.00

Course book

In the course we use a book. You have already received it in the welcome email, but you can also download it from DTU-Learn. This material is written for this course. We have carefully proof read it, but we may missed a few places. Please email us if you find errors, and we will update the book.

Course book II

The book contains:

- **Chapter 1:** A course plan detailing which assignments to solve in the period 5/1 - 22/1.
- **Chapter 2:** A short chapter on the installation of Julia, JuMP, solvers and editors.
- **Chapter 3:** A chapter with a short Linear Programming (modeling) introduction.
- **Chapter 4:** 6 different Linear Programming assignments.
- **Chapter 5:** A chapter with a short Mixed Integer Programming (modeling) introduction.
- **Chapter 6:** 23 different Mixed Integer Programming assignments.
- **Chapter 7:** A chapter with a short Multi-objective Mixed Integer Programming (modeling) introduction.
- **Chapter 8:** 4 different Multi-objective Mixed Integer Programming assignments.

Study Groups

You can work in groups of several students:

- It is, for many of you, a good idea to work in groups ...
- ... but the final exam is individual, so make sure that you do not just follow along. You have to **understand** the models
- If you prefer, you can work alone, you do not **have** to be in a group, your choice.

Important: You are yourself responsible for getting into a group.

Assignment answers

- We will upload a pdf file with a detailed description of the correct models for the days assignments, 17.00 every course day.
- The pdf file also contains correct Julia/JuMP models

This course is not

This course is not:

- A programming course: Julia **is** a general programming language ...
- ... but in this course, we will only use a small limited part of the language (in particular for math heuristics)

This course is a **MODELING COURSE**. Hence the focus is mostly on formulating mathematics, not on computer programming.

However, we will experiment this year to provide you with some more information on how to program these models – we look forward to receive your feedback.

Julia and the Math Modeling course

We will use the Julia programming language and the JuMP modeling package extensively:

- If you have never used Julia/JuMP before, you may have to work harder ...
- If you have not installed Julia before, it is the first thing you do after the lecture ...

What is Mathematics

- Mathematics is a beautiful world of its own
- and a powerful tool which is used far to little !!!

Hopefully this course will show both features: Beautiful models and (very) powerful tools which may be used to solve hard problems.

Models

Assume you got the question: How long time will it take to drive from Copenhagen to Paris ?

A traveling time model could look something like:

$$Time = \frac{Distance}{Avg.speed}$$

- This model is so simple that you often do not consider it a model !
- A large number of details are left out ...
- You often **assume** that the minimum time is of interest, hence it is actually an optimization problem ... with just one variable: Speed.

Models

Why is modeling interesting ?

- **Modeling is understanding:** If you cannot model a problem, I doubt you understand it !
- **Modeling is communication:** A complex mathematical model can communicate problems in the language of mathematics.
- **Modeling is getting solutions:** Companies are generally not very interested in models, they are interested in solutions, but using general solvers we can get that !

What is an Optimization model

In this course a model consists of three parts:

- One or more (decision) variables
- One (**or more**) objective function
- A number of constraints

We will work with three different types of models: Linear Programming (LP) models, Mixed Integer Programming (MIP) models and Multi-Objective Mixed Integer Programming (MOMIP) models.

We will start with LP models the first 2 days of the course and then consider MIP models and MOMIP models the rest of the course.

Example: Incredible Chairs

This example is explained in detail in Chapter 3 of the book.

The company Incredible Chairs produces two different types of chairs, A and B , where one unit (pallet of chairs) of Chair A can be sold for a profit of 4 and one unit of Chair B can be sold for a profit of 6. The chairs are produced on three production lines:

- 1 Line 1 requires 2 hour to produce one unit of Chair A , and at most 14 hours can be used.
- 2 Line 2 requires 3 hours to produce one unit of Chair B , and at most 15 hours can be used.
- 3 Line 3 requires 4 hours to produce one unit of Chair A and 3 hours to produce one unit of chair B , and at most 36 hours can be used.

A direct LP program for Incredible Chairs

The complete LP model is therefore:

$$\begin{array}{llllll} \text{Maximize.} & 4 \cdot x_A & + & 6 \cdot x_B & & \\ \text{Subject to :} & & & & & \\ & 2 \cdot x_A & & & \leq & 14 \\ & & & 3 \cdot x_B & \leq & 15 \\ & 4 \cdot x_A & + & 3 \cdot x_B & \leq & 36 \\ & x_A & & & \geq & 0 \\ & & & x_B & \geq & 0 \end{array}$$

Incredible Chairs in Julia/JuMP (Compact Version)

```
using JuMP
using HiGHS
IC = Model(HiGHS.Optimizer)

@variable(IC, xA >= 0)
@variable(IC, xB >= 0)

@objective(IC, Max, 4*xA + 6*xB)
@constraint(IC, 2*xA <= 14)
@constraint(IC, 3*xB <= 15)
@constraint(IC, 4*xA + 3*xB <= 36)

print(IC)
optimize!(IC)
println("Termination status: $(termination_status(IC))")
if termination_status(IC) == MOI.OPTIMAL
    println("Optimal objective value: $(objective_value(IC))")
    println("xA: ", value(xA))
    println("xB: ", value(xB))
else
    println("No optimal solution available")
end
```

Incredible Chairs in Julia/JuMP, standard layout I

Naming and introductory definitions:

```
#####  
# Incredible Chairs, Simple LP  
using JuMP  
using HiGHS  
#####
```


Incredible Chairs in Julia/JuMP, standard layout II

Data definitions:

```
#####  
# PARAMETERS  
Chairs = ["A", "B"]  
C=length(Chairs)  
ProductionLines = ["1","2","3"]  
P=length(ProductionLines)  
  
Profit = [4,6]  
Capacity = [14,15,36]  
RecourseUsage =  
    [  
        2 0;  
        0 3;  
        4 3  
    ]  
#####
```

Incredible Chairs in Julia/JuMP, standard layout III

Model definition:

```
#####  
# Model  
IC = Model(HiGHS.Optimizer)  
  
@variable(IC, x[1:C] >= 0)  
  
@objective(IC, Max, sum( Profit[c]*x[c] for c=1:C))  
  
@constraint(IC, [p=1:P], sum( RecourseUsage[p,c]*x[c] for c=1:C) <= Capacity[p])  
#####
```

Incredible Chairs in Julia/JuMP, standard layout III

Send the model to the Solver:

```
#####  
# Solve  
optimize!(IC)  
println("Termination status: $(termination_status(IC))")  
#####
```

Incredible Chairs in Julia/JuMP, standard layout IIII

Print out solution:

```
#####  
# Solution  
if termination_status(IC) == MOI.OPTIMAL  
    println("Optimal objective value: $(objective_value(IC))")  
    for c=1:C  
        println("x[" , Chairs[c], "] : ",value(x[c]))  
    end  
else  
    println("No optimal solution available")  
end  
#####
```

Copying previous models

There is no problem in copying the IncredibleChairs.jl program for the next assignments, however:

- Be carefull
- Make sure to name the Sets, Cardinaltiy values and index names correctly:
 - Here the sets are "Chairs" and "ProductionLines".
 - The cardinality values C and T.
 - And the indexes are c and t

Software: Julia/JuMP/HiGHS/VS-studio

The software we recommend to use in the course is:

- Julia: General open source language (think fast Python). We use version 1.11 or newer (but which version you use is not critical).
- JuMP: The mathematical programming package, very easy to install using:
- HiGHS: Open source solver, very easy to install using:
- Visual Studio: The editor to use to write programs (you can choose another if you wish)

To install all this, go to the following link:

`https://code.visualstudio.com/docs/languages/julia`

Julia

Julia is an open source language out of MIT, which we use in all OR courses at DTU. The newest stable version is 1.12. Version 1.10 and later is fine. With the above link, you can find a direct link to go and install it. It is directly downloadable, and since it is open source, easy to install.

JuMP

JuMP is the Mathematical Programming package for Julia. In our/my opinion, this is the best modeling package for LP/MIP/MOMIP in the world. Using the Julia package system, it is very easy to install:

```
import Pkg  
Pkg.add("JuMP")
```


HiGHS

HiGHS is the solver we mostly use to get the solutions to our models. There are alternatives, mentioned in the next slides, but HiGHS is open source (meaning you can use it afterwards for anything you want), and it is very easy to install:

```
import Pkg  
Pkg.add("HiGHS")
```

Notice, that for some of the assignments, e.g. University Timetabling, HiGHS will be challenged/slow.

VS-studio

To write Julia programs/models, we recommend you use the VS-studio, **but** you can in principle use any text editor you want (I am using the rather old-school Emacs editor).

To install all this, go to the following link:

<https://code.visualstudio.com/docs/languages/julia>

Commercial solvers

For Mixed Integer Linear Programming (MIP), **either:**

- Gurobi (v. 13.00) (may be installed with academic license when you are logged into DTU Wifi installation):

`https://www.gurobi.com/
downloads/end-user-license-agreement-academic/`

- IBM ILOG CPLEX Optimization Studio 20.1.0, you can have it for free, but is a lot of work:

`https://community.ibm.com/community/user/
datascience/blogs/xavier-nodet1/
2020/07/09/cplex-free-for-students`

When you read a new assignment, think:

- **Which decision variables do I need:** What do I control ? (In incredible chairs you control how many of each chair to produce)
- Sometimes it is convenient to define **auxilliary** variables, i.e. variables which are fully defined by other variables but which may make the model simpler.
- Given the variables, how can I define the **objective function** to calculate the value to be optimized given the decision variable values
- Given the variables, how can I use **constraints** to define the requirements (constraints) of the model.

DTU



Lecture 2: Basic JuMP

Mathematical Programming Modelling

Lecture 2 : Basic JuMP

- A few comments on our use of Julia in this course
- Full definition of @objective
- Full definition of @variable
- Full definition of @constraint
- Data
- Comparison of JuMP LP model and Mathematical Programming LP model

Julia in this course

This course is **not** a programming course. In fact we are doing our best to keep the programming part as simple as possible, **but** Julia is a full-blown programming language which can be used for basically any programming problem. In the OR section at Management Science division at DTU-Management Engineering, we use Julia and JuMP in (almost) all our courses.

Julia Simplifications

- One file
- No use of functions necessary (but if you want to use them be my guest ...)
- Only use of few packages:
 - JuMP : The modelling package, which enables easy formulation of Linear Programs (LP) and Mixed Integer Programs (MIP)
 - Solvers: HiGHS. If you want to use Gurobi go ahead, but it may be more complicated to install
 - Printf : Printing package. This is not strictly necessary.

First time you try to use JuMP, HiGHS, or Printf, you will have to import it, e.g.: `"import Pkg; Pkg.add("HiGHS")"`. This works immediately for HiGHS, and Printf, but only works for Gurobi if you have already downloaded and installed it. Depending on your computer, this can be complicated

Data

In this course we only need two types of data: Text, Booleans and (mostly) numbers. We in general need these data in different dimensions:

- Number (Zero dimensional)
- One dimensional vector
- Two dimensional vector
- Three dimensional vector

Most often, text is only necessary in one-dimensional vectors.

Text vectors

Examples:

```
names = ["Alice", "Bob", "Charlie", "Doris"]  
N=length(names)
```

Numbers (zero dimensional)

Examples:

$f=90$

$Q=31$

One dimensional

Examples:

```
customer_orders=[11 7 6 12]  
C=length(customer_orders)
```

Two dimensional

Examples:

```
distances=[
17  39  84 ;
109  32  98 ;
139   4  31 ;
123 129 116 ]
(I,J)=size(distances)
```

Three dimensional

```
time_distances=zeros(Int16,2,4,3)
time_distances[1,:,:]=[
17  39  84 ;
109 32  98 ;
139  4  31 ;
123 129 116 ]
time_distances[2,:,:]=[
37  49  54 ;
609 72  88 ;
939  1  21 ;
133 149 156 ]
(T,I,J)=size(time_distances)
```

A very important point

In almost all our models, we have data over **SETS**, e.g. sets of products, production-lines, depots, persons, cities etc.:

```
Persons = ["Alice", "Bob", "Charlie", "Doris"]  
P=length(Persons)
```

- Notice that just beneath the vector, we find **the number of Persons P**
- This number P is hugely important: The indexes is 1 to P
- In a number of constructs, we then define idx variables, examples:
 - for p = 1:P
 - @constraint(m, balance_con[p=2:P],
 - @variable(m, x[1:P,1:P] >= 0)

Model statement: Different solvers

```
m = Model(HiGHS.Optimizer)
```

```
m = Model(Gurobi.Optimizer)
```

variables I: Single variables

```
@variable(m, x )           # No bounds  
  
@variable(m, lb <= x )     # Lower bound only  
  
@variable(m, x <= ub )     # Upper bound only  
  
@variable(m, lb <= x <= ub ) # Lower and upper bounds
```

variables II: Vector of variables

```
@variable(m, 0 <= x[1:M,1:N])
```

```
@variable(m, 0 <= x[i=1:10,j=i:10])
```

Variables: Domain bounds

It is possible to create more complex bounds, example:

```
@variable(m, x[i=1:I])  
for i=1:I  
    if mod(i,3)==0 # insert any expression here  
        set_lower_bound(x[i],0)  
        set_upper_bound(x[i],0)  
    end  
end
```

Variables: Domain bounds

Alternatively:

```
@variable(m, x[i=1:I])  
for i=1:I  
    if mod(i,3)==0 # insert any expression here  
        fix(x[i],0; force = true)  
    end  
end
```

Objective

```
@objective(m, Min,  
sum(distance[i,j]*x[i,j] for i=1:I, j=1:J))
```

```
@objective(m, Min,  
sum(distance[i,j]*x[i,j] for i=1:I, j=1:J if i < j))
```

Constraints

```
@constraint(m, con, x + y <= 1)
```

```
@constraint(m, [i=2:I,j=1:J; i!=j],  
            (I-1)*x[i,j] + u[i] - u[j] <= (I-2) )
```

```
@constraint(m, [i=2:I],  
            sum(x[j,i] for j=1:I if j!=i) ==  
            sum(x[i,j] for j=1:J if j!=i) )
```

Incredible Chairs 2: Math. Model - Julia Model

```
@variable(IC,x[c=1:C] >=0 )  
@objective(IC, Max, sum( Profit[c]*x[c])  
@constraint(IC, [p=1:P],  
sum( RecourseUsage[p,c] * x[c] for c=1:C) <= Capacity[p])
```

Maximize:
$$\sum_{c \in C} Profit_c \cdot x_c$$

Subject to :

$$\sum_{c \in C} RecourseUsage_{p,c} \cdot x_c \leq Capacity_p \quad \forall p \in P$$

$$x_c \geq 0 \quad \forall c \in C$$

A very important and slightly complex construct

In Julia it is possible to write the following expression:

```
(i>1 ? y[i-1] : 0)
```

This is an in-line if statement which can be used in e.g. a constraint: **if** $i > 1$, insert $y[i-1]$, otherwise ($i \leq 1$), then insert 0.

Notice that i is an index, which always is an integer number $1 \leq i \leq I$.

In many models, we need to relate "neighbour" variables, e.g. timeslots. Here timeslot i is related to the previous timeslot $i - 1$. This construct enables to handle border cases $i = 1$. This construct you need to use the first time in the Micro Brewery assignment.

Including a datafile

If you have a file with data, written in Julia (which you need in a number of assignments like Stamps), you can use the command:

```
include("stamp_bid_data.jl")
```

Notice you may have to give the full path to the datafile.
Alternatively, you can simply copy the data into the same file.

Why Julia ?

We now use Julia in all OR courses, why ?

- By using Julia/JuMP in all courses, we expect students have a better basis for a master thesis
- Julia/JuMP is the first programming language which can cater **ALL** our needs
- The JuMP package is very good

Why Julia ?

- **Julia/JuMP is open-source:** Which means that you can legally use Julia/JuMP together with an open-source solvers e.g. HiGHS (**not** Gurobi), without breaking the law and without paying any licenses.
- Julia is a state-of-art new programming language which is "as fast as C and as easy as Python". And we **HAVE** checked the speed, it can really be as fast as C.

Notice, there are a number of alternative specialized Mathematical Programming languages: GAMS, AIMMS, AMPL, OPL-Solver ...

DTU



Mixed Integer Programming (MIP)

Lecture 3

Learning Objectives

- Know what Mixed Integer Programming (MIP) is
- Understand why this type of model is (solution-wise) much more complex
- Know how to model a fixed cost

First a bit of OR history

- The term Operations Research is invented in 1938
- OR is used during the second world war to direct convoys
- First linear program suggested in 1945 (probably earlier in the Soviet Union ...)
- The Simplex algorithm was invented by George Dantzig in 1947
- Around the 1960s an issue arises: some problems are **inherently** integer. This leads to the development of:
 - Branch & Bound algorithms
 - Branch & Cut algorithms
 - Decomposition algorithms

Applications of integer optimization

- Airline industry (crew scheduling)
- Telecommunications (network planning)
- Electricity planning (which plants to run)
- Cutting problems (how to cut paper, carpets, etc)
- Many more.....

Used to save millions of \$s every day.

Applications of integer optimization at DTU

- Container ship routing (with Mærsk)
- Loading of container ships
- Airport optimization (Copenhagen Airport, Frankfurt airport)
- Timetabling (Macom, DTU, DSB)
- Man-power planning (Nurse rostering at Region Zealand, QAMPO)
- Line Planning, Rolling Stock optimization, (DSB)
- Vehicle Scheduling (AMCS, QAMPO)

Basic integer optimization problem

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \\ & x_j \geq 0 \quad \forall j \\ & x \in \mathbb{Z}^n \end{aligned}$$

Basic integer optimization problem

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \\ & x \in \{0, 1\}^n \end{aligned}$$

Integer Variables in JuMP

```
@variable(m, x[1:M,1:N], Bin)
```

```
@variable(m, x[i=1:10,j=i:10], Int)
```

```
@variable(m, 0 <= x[i=1:10,j=i:10] <= 13, Int)
```

Notice: If you do not write types of the variables, they are assumed to be continuous, inside a lower and upper bound. Then you can simply add either **Bin** or **Int**. For the integer variables you can also add lower and upper bounds. This is obviously not necessary for **Bin** variables where the lower bound is 0 and 1.

Classic OR models LP

There are a number of classic OR models which are good to know:

- **Assignment:** Assign N workers to N jobs (LP, polynomial time complexity)
- **Shortest path:** Find the shortest path through a network (LP, polynomial time complexity)
- **Multi-Commodity Flow:** Find the cheapest flow of multi-commodities through a network (LP, polynomial time complexity)

Classic OR models MIP

- **Knapsack:** Select the most profitable items given a capacity constraint
- **Set-cover, Set-packing & Set-partitioning:** Given a set of items, and sets of sets of items: Either pack, cover or partition the sets of sets of items
- **Lot Sizing:** Produce to order and storage
- **Travelling Salesman & Vehicle Routing:** Route one truck or several trucks as cheaply as possible
- **Facility Location:** Place facilities to reduce investment and transport cost

Assignment problem

- Given n jobs
- Given n persons
- Given c_{ij} cost of assigning job i to person j

Model:

- Decision variables:

$$x_{ij} = \begin{cases} 1 & \text{person } i \text{ is assigned job } j \\ 0 & \text{otherwise} \end{cases}$$

- Objective to be minimized:

$$\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

- Each project must be assigned one person:

$$\sum_{j=1}^n x_{ij} = 1 \quad \forall i$$

- Each person must be assigned to one project:

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j$$

Solution approaches:

- Replace integer restriction by the LP restriction:

$$0 \leq x_j \leq 1$$

- Is that a good idea?

- Round the solution
- Try all feasible solutions
- Using integer optimization algorithms:
 - Branch and bound
 - Cutting plane algorithms
 - Heuristics ...
- Fact: **In general integer optimization problems are very difficult to solve**
- The assignment problem is easy!
- Remember the Chair Transportation problem? There you got integer solutions, even though it was an LP model

The Knapsack Problem

- Given n projects which each require a_j \$ of capital and give a return c_j
- A limited budget b \$ is available
- Only whole projects make sense

Model:

- Decision variables:

$$x_j = \begin{cases} 1 & \text{project } j \text{ is chosen} \\ 0 & \text{otherwise} \end{cases}$$

- Objective to be maximized:

$$\sum_{j=1}^n c_j x_j$$

- Budget constraint:

$$\sum_{j=1}^n a_j x_j \leq b$$

The Set Covering Problem

- Assume you have m regions
- Build new fire stations so that each region can be reached within certain time limit say 5 minutes. $a_{ij} = 1$ if that is the case. Otherwise $a_{ij} = 0$
- c_j is cost associated with building fire station of j

Model:

- Decision variables:

$$x_j = \begin{cases} 1 & \text{fire station } j \text{ is build,} \\ 0 & \text{otherwise} \end{cases}$$

- Objective to be minimized:

$$\sum_{j=1}^n c_j x_j$$

- Coverage constraints:

$$\sum_{j=1}^n a_{ij} x_j \geq 1 \quad \forall i$$

- Important problem class
- Wide usage in the airline industry

The Traveling Salesman Problem

- A salesperson must visit n cities
- Each city must only be visited once
- The tour must start and end at the same city
- c_{ij} is the travel time (or cost) between city i and j

Model:

- Decision variables:

$$x_{ij} = \begin{cases} 1 & \text{travel from city } i \text{ to city } j \text{ directly.} \\ 0 & \text{otherwise} \end{cases}$$

(x_{ii} are left undefined)

- Objective to be minimized:

$$\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

- The tour leaves each city i exactly once:

$$\sum_{\substack{j=1 \\ j \neq i}}^n x_{ij} = 1 \quad \forall i$$

- The tour arrives at each city j exactly once:

$$\sum_{\substack{i=1 \\ i \neq j}}^n x_{ij} = 1 \quad \forall j$$

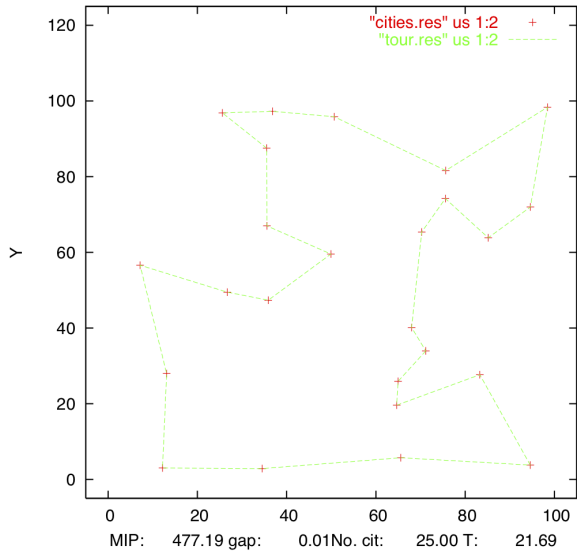
- Are these the only necessary constraints?
- Subtour elimination constraints must also be included:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 1 \quad \forall S$$

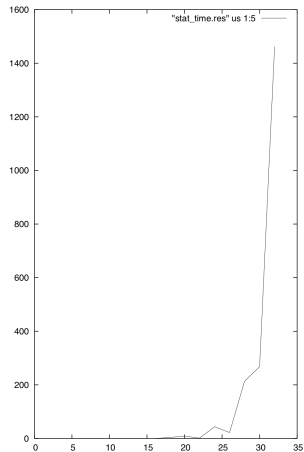
S is an arbitrary subset of all cities

- Very useful problem
- Very difficult problem $((n-1)!$ feasible solutions)
 - Observe the many constraints

TSP



TSP optimization time



Modeling fixed cost

Examples:

- Startup cost
- Fixed transactions cost in investment
- Facility location

Mathematical description:

$$h(x) = \begin{cases} 0, & x = 0 \\ f + px, & x > 0 \end{cases}$$

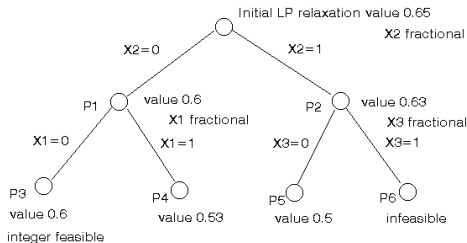
Model:

- Assume $x \leq B$ (B is some upper bound).
- Mathematical reformulation:

$$\begin{aligned} h(x) &= fy + px, \\ x &\leq By, \\ y &\in \{0, 1\}. \end{aligned}$$

Solving MIP models

The standard approach to solve Mixed Integer Programs is the Branch & Bound algorithm:



Making solvers for LP and (especially) MIP is rocket-science!

Solver speed-up

One of the main reasons for Mathematical Programming to become usable, is the solver speedup (and hardware speedup). This goes for both Linear Programming and Mixed Integer Programming.

The stats here for the period 2001-2020 are from the paper: "Progress in mathematical programming solvers from 2001 to 2020", Thorsten Koch, Timo Berthold, Jaap Pedersen % Charlie Vanaret, EURO Journal on Computational Optimization 10 (2022) (you can find it on Google Scholar)

Solver speed-up: Linear Programming

"For lps, we computed an average speed-up factor of nine. Combining this with the hardware speed-up, we can conclude that solving lps got about 180 times faster in the last two decades. However, the main difference comes from the switch to 64-bit computing, allowing to solve much larger instances, in particular with parallelized barrier codes. Furthermore, it is fair to say that the solver implementations became ever more refined, leading to extremely stable codes. At the same time, little progress has been made on the theoretical side."

Solver speed-up: Linear Programming

"In this case, the picture gets more diverse. For the instances solved with the old codes, the average speed-up due to improved algorithms (Fig. 1) is about 50. This means that solving milps got faster by 22 % every year during the last 20 years, purely from the algorithmic side, and that is not taking into account how many more instances we can solve today as compared to twenty years ago. Combining this with the hardware speed- up, we find an average total speed-up for solving milps of 1,000: fifteen minutes then become less than a second now."

Solvers

In this course we recommend to use one of two solvers:

Then the type of solvers:

- HiGHS Open-Source
- Gurobi Commercial

When we are solving LP, the difference is small, but it is **NOT** the case for MIP. You will see the difference clearly in the problems Student Teacher Meetings and Groceries.

Solvers: There are many more

The current stable version of JuMP supports **41** different solvers, which can be used for a variety of problem types:

- LP = Linear programming
- QP = Quadratic programming
- SOCP = Second-order conic programming (including problems with convex quadratic constraints or objective)
- MCP = Mixed-complementarity programming
- NLP = Nonlinear programming
- SDP = Semidefinite programming
- (MI)XXX = Mixed-integer equivalent of problem type XXX

We only consider LP and MIP. However, Gurobi can handle general QP models. This can be a problem, because if you formulate such a model by accident you may not even realize it ...

Two nice JuMP options

Through JuMP we can control (generically for all solvers):

- Printout in execution window:

```
set_silent(model)
unset_silent(model)
```

- The execution time, however only the best found solution is returned (if any is found), optimality is not guaranteed.

```
set_time_limit_sec(model, 100)
unset_time_limit_sec(model)
```

DTU



Mixed Integer Programming Hardness

Lecture 4

Learning Objectives

This lecture introduces:

- Two types of variables
- Why MIP models are hard
- What can be done to address this?

Variables

There are two types of variables in your Julia/JuMP code:

- Julia variables:

```
i=50000
```

- JuMP variables (decision variables):

```
@variable(knapsack, x, Bin)
```

Notice: You cannot use JuMP variables (decision variables) in the program flow code, e.g. if statements.

Logic OR example (Incorrect)

```
@variable(knapsack, x, Bin)
@variable(knapsack, y, Bin)
@variable(knapsack, v, Bin)
```

```
if x==1
    v=1
end
if y=1
    v=1
end
if x==0 && v==0
    v=0
end
```

Logic OR example (Incorrect)

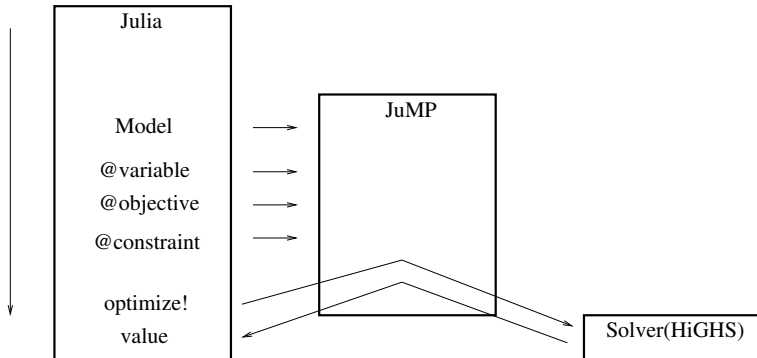
```
@variable(knapsack, x, Bin)
```

```
@variable(knapsack, y, Bin)
```

```
@variable(knapsack, v, Bin)
```

```
@constraint(v == (x==1 || y==1 ? 1 : 0) )
```

Variables: Execution from top to bottom



We cannot use the values of the decision variables before solving the problem because they are not known until the model has been solved

Constraint implementation: Logic OR example ($v = x \vee y$)

Correct:

```
@variable(knapsack, x, Bin)
@variable(knapsack, y, Bin)
@variable(knapsack, v, Bin)

@constraint(knapsack, v >= x)
@constraint(knapsack, v >= y)
@constraint(knapsack, v <= x+y)
```

Wrong:

```
@variable(knapsack, x, Bin)
@variable(knapsack, y, Bin)
@variable(knapsack, v, Bin)

if x==1
    v=1
end
if y==1
    v=1
end
if x==0 && v==0
    v=0
end
```

Using JuMP variables

- You cannot **set** the JuMP variables:

```
v=1
```

The value of JuMP variables are set by the **solver**

- You cannot read (directly) the value of a JuMP variable:

```
if x==1
```

After the optimize! statement, the solver has set values of the variables and we can get them and use them:

```
if value(x)==1
```

What does a MIP consist of ?

- An objective function
- A number of constraints
- A (possible) number of continuous variables
- A number of integer or binary variables

Abstract and Direct

In a MIP model we can talk about **abstract** and **direct** variables or constraints:

- One **abstract** constraint (from Stamps):

```
@constraint(stamps, [s=1:S],  
            sum( BidSets[b,s]*x[b] for b=1:B) <= 1 )
```

leads to S direct constraints (100)

- One **abstract** variable (from Stamps):

```
@variable(stamps, x[1:B], Bin)
```

leads to B direct variables (213)

The **size** of a model is the number of direct constraints and variables.

Hard MIP

There are three main problems:

- The number of **direct** binary/integer variables
- A large *gap*
- Symmetries (not discussed here ...)

What can be done about these things?

MIP relaxation

Given a MIP:

$$\min z = \sum_{j=1}^n c_j x_j \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \{1, \dots, m\}, \quad (2)$$

$$x_j \geq 0, \quad \forall j \in \{1, \dots, n\}, \quad (3)$$

$$x_j \text{ integer} \quad \forall j \in \{1, \dots, n\}. \quad (4)$$

MIP relaxation

The LP relaxation is:

$$\min z = \sum_{j=1}^n c_j x_j \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \{1, \dots, m\}, \quad (2)$$

$$x_j \geq 0 \quad \forall j \in \{1, \dots, n\} \quad (3)$$

$$(4)$$

MIP relaxation 2

$$\min Z_{LP} = x_1 + 10x_2$$

$$\text{s.t. } 2x_1 + x_2 \geq 8$$

$$1 - 3x_1 + x_2 \geq -12$$

$$x_1 - 4x_2 \geq -20$$

$$-x_1 + 2x_2 \geq -1$$

$$x_1, x_2 \in \mathbb{R}$$

$$\min Z_{IP} = x_1 + 10x_2$$

$$\text{s.t. } 2x_1 + x_2 \geq 8$$

$$-3x_1 + x_2 \geq -12$$

$$x_1 - 4x_2 \geq -20$$

$$-x_1 + 2x_2 \geq -1$$

$$x_1, x_2 \in \mathbb{Z}$$

Which one is true?

① $Z_{LP} \leq Z_{IP}$

② $Z_{LP} \geq Z_{IP}$

③ we cannot tell without solving the two problems.

MIP relaxation 2

$$\min Z_{LP} = x_1 + 10x_2$$

$$\text{s.t. } 2x_1 + x_2 \geq 8$$

$$1 - 3x_1 + x_2 \geq -12$$

$$x_1 - 4x_2 \geq -20$$

$$-x_1 + 2x_2 \geq -1$$

$$x_1, x_2 \in \mathbb{R}$$

$$\min Z_{IP} = x_1 + 10x_2$$

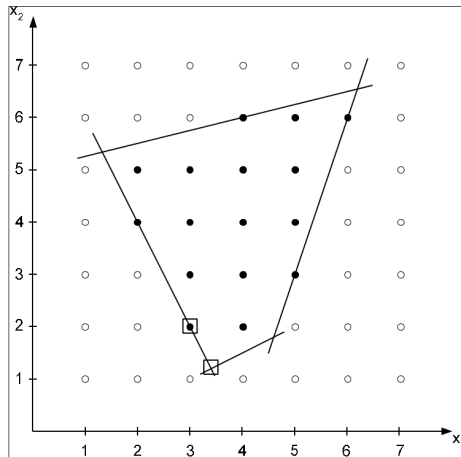
$$\text{s.t. } 2x_1 + x_2 \geq 8$$

$$-3x_1 + x_2 \geq -12$$

$$x_1 - 4x_2 \geq -20$$

$$-x_1 + 2x_2 \geq -1$$

$$x_1, x_2 \in \mathbb{Z}$$



LP sol: $x_1=3.4$, $x_2=1.2$, $Z=15.4$

IP sol: $x_1=3$, $x_2=2$, $Z=23$

The Gap

$$\min z = \sum_{j=1}^n c_j x_j \quad (5)$$

$$\text{s.t.} \quad \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \{1, \dots, m\}, \quad (6)$$

$$x_j \geq 0, \quad \forall j \in \{1, \dots, n\}, \quad (7)$$

$$x_j \text{ integer} \quad \forall j \in \{1, \dots, n\}. \quad (8)$$

Assume that the objective value of the optimal solution to this problem is $\bar{z}$. The LP relaxation is the problem obtained by removing the integrality requirements. All variables are continuous (inside the same bounds). The optimal objective of the relaxed problem is: $\underline{z}$. We know that $\underline{z} \leq \bar{z}$ (the relaxed solution provides a lower bound on the possible optimal value). The gap is : $gap = \bar{z} - \underline{z}$. If the gap is large, the problem is hard.

The number of binary/integer variables

Worst case, the solver has to check possible integer solutions. Hence, if there are N binary variables, there are 2^N possible solutions:

- Notice that the number of possible solutions is directly dependent on the number of integer variables.
- Given N integer variables, the number of possible solutions is

$$\prod_{i=1}^N (N_i^{high} - N_i^{low} + 1)$$

The Branch & Bound tree 1

Solve LP relaxation -> Lower bound (LB)

Find feasible solution to IP -> Upper bound (UB)



LB=5429.0
UB=?

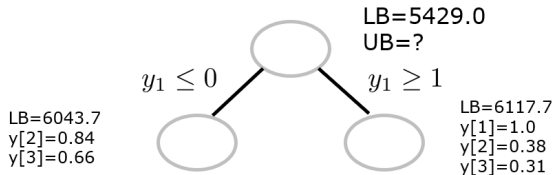
$y[1]=0.52$
 $y[2]=0.58$
 $y[3]=0.40$

- We cannot decide which solution is optimal based on the available information.
- Split solution space in two:
- Find a variable that takes a non-integer value in the LP solution. Let's use y_1 .
- One sub-problem is constructed by adding $y_1 \leq 0$ to the set of constraints. The other sub-problem is constructed by adding $y_1 \geq 1$ to the set of constraints.

The Branch & Bound tree 2

Solve LP relaxation -> Lower bound (LB)

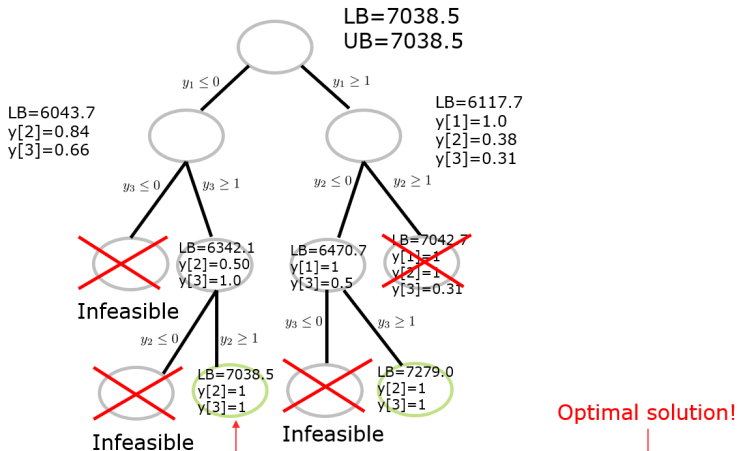
Find feasible solution to IP -> Upper bound (UB)



The Branch & Bound tree 3

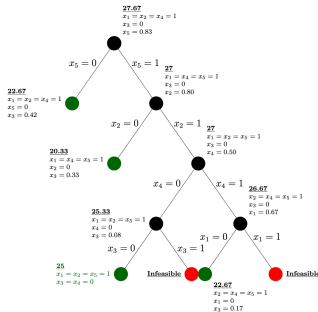
Solve LP relaxation -> Lower bound (LB)

Find feasible solution to IP -> Upper bound (UB)



The Branch & Bound tree 4

The Branch & Bound algorithm is explained in some detail in Chapter 5:



Note: The depth of the tree depends on the number of **Integer/Binary** variables.

Reduce the number of Integer/Binary variables

Hence: **Reduce the number of Integer/Binary variables as much as possible.:**

- Some variables are naturally integer, do not declare them to be integer
- Use the FIX command (Lecture 3) to fix as many Integer/Binary variables to a fixed value (often zero) as possible

```
fix(x[i],0; force = true)
```

MIP Gap

Gurobi and HiGHS calculates the MIP Gap:

$$\text{MIP Gap} = \frac{|BestBd - Incumbent|}{|Incumbent|}$$

A few comments regarding the MIP Gap:

- A Branch & Cut algorithm uses branching, and cutting, to reduce the MIP Gap
- For a MIP, if there is a large gap between the objective value of an optimal solution to the LP relaxation and the objective value of an optimal integer solution, then lots of branching may be required
- This may be time consuming ...

Gurobi 10.0 print-out (during execution)

Root relaxation: objective 1.445964e+03, 100 iterations, 0.00 seconds (0.00 work units)

Nodes		Current Node			Objective Bounds			Work	
Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
0	0	1445.96429	0	48	-	1445.96429	-	-	0s
0	0	1527.98469	0	52	-	1527.98469	-	-	0s
0	0	1527.98469	0	52	-	1527.98469	-	-	0s
0	0	1580.00000	0	30	-	1580.00000	-	-	0s
0	0	1580.00000	0	58	-	1580.00000	-	-	0s
0	0	1583.00000	0	54	-	1583.00000	-	-	0s
0	0	1583.00000	0	54	-	1583.00000	-	-	0s
0	0	1583.00000	0	54	-	1583.00000	-	-	0s
0	0	1583.00000	0	54	-	1583.00000	-	-	0s
0	0	1583.00000	0	54	-	1583.00000	-	-	0s
H	0	0			1917.0000000	1583.00000	17.4%	-	0s
	0	0	1583.00000	0	54	1917.00000	1583.00000	17.4%	0s
	0	0	1583.00000	0	54	1917.00000	1583.00000	17.4%	0s
	0	0	1583.00000	0	54	1917.00000	1583.00000	17.4%	0s
	0	2	1583.00000	0	54	1917.00000	1583.00000	17.4%	0s
H	37	40			1848.0000000	1584.50893	14.3%	13.3	0s
H	65	71			1702.0000000	1584.50893	6.90%	13.1	0s
*	1874	1072		22	1625.0000000	1596.35857	1.76%	10.3	1s
*	1939	992		21	1610.0000000	1596.35857	0.85%	10.3	1s

Gurobi 10.0 print-out (after execution)

Cutting planes:

Learned: 13

Gomory: 72

Implied bound: 14

MIR: 11

Flow cover: 32

Inf proof: 5

Zero half: 1

Explored 2256 nodes (22308 simplex iterations) in 1.80 seconds (1.49 work units)

Thread count was 16 (of 16 available processors)

Solution count 5: 1610 1625 1702 ... 1917

Optimal solution found (tolerance 1.00e-04)

Best objective 1.610000000000e+03, best bound 1.610000000000e+03, gap 0.0000%

User-callback calls 5665, time in user-callback 0.01 sec

Gurobi printout notice

- Look at the GAP column: At first there is no gap, just a dash ("-"), because there is no incumbent solution
- When the GAP reaches 0, the program terminates
- During the execution both the Incumbent and the Best Bd (Best Bound) improve. Often the big jumps are in the Incumbent
- The status of the tree search can be seen in the first two columns:
Nodes : Expl Unexpl:
 - Unexpl: The number of active nodes that remain to be solved. When this number reaches 0 the algorithm terminates
 - Expl: The number of nodes which have already been explored
- Upon termination, Gurobi states the number of explored nodes 2256 (many were explored in the end and simply deleted) and lists briefly the objective of the found incumbents: 1610 1625 1702 ... 1917

HiGHS print-out (during execution)

	Nodes		B&B Tree		Objective Bounds		Dynamic Constraints				Work	
	Proc.	InQueue	Leaves	Expl.	BestBound	BestSol	Gap	Cuts	InLp	Confl.	LpIters	Time
	0	0	0	0.00%	0	inf	inf	0	0	0	0	0.0s
	0	0	0	0.00%	1445.964286	inf	inf	0	0	0	90	0.0s
L	0	0	0	0.00%	1593	1866	14.63%	2605	116	36	626	3.1s
L	0	0	0	0.00%	1593	1695	6.02%	2605	116	36	1839	4.6s
45.4% inactive integer columns, restarting												
Model after restart has 638 rows, 471 cols (443 bin., 0 int., 0 impl., 28 cont.), and 2449 nonzeros												
	0	0	0	0.00%	1593	1695	6.02%	38	0	0	3085	4.6s
	0	0	0	0.00%	1593	1695	6.02%	38	17	6	3137	4.6s
L	0	0	0	0.00%	1593	1654	3.69%	1897	111	6	3385	6.1s
L	0	0	0	0.00%	1593	1635	2.57%	2044	92	6	4498	7.1s
51.7% inactive integer columns, restarting												
Model after restart has 534 rows, 242 cols (214 bin., 0 int., 0 impl., 28 cont.), and 1576 nonzeros												
	0	0	0	0.00%	1593	1635	2.57%	37	0	0	6194	7.1s
	0	0	0	0.00%	1593	1635	2.57%	37	22	8	6252	7.1s
B	66	9	26	45.51%	1593	1625	1.97%	2222	35	2035	19681	10.1s
B	94	5	40	75.78%	1593	1610	1.06%	2263	35	2561	22140	10.6s

HiGHS print-out (after execution)

Solving report

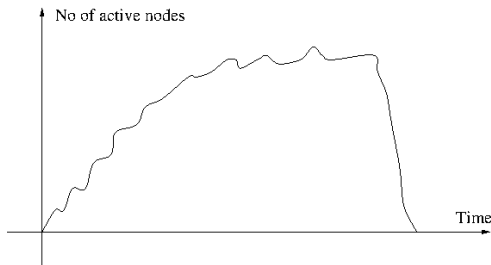
Status	Optimal
Primal bound	1610
Dual bound	1610
Gap	0% (tolerance: 0.01%)
Solution status	feasible
	1610 (objective)
	0 (bound viol.)
	1.09753476149e-14 (int. viol.)
	0 (row viol.)
Timing	11.01 (total)
	0.03 (presolve)
	0.00 (postsolve)
Nodes	124
LP iterations	24835 (total)
	14229 (strong br.)
	1249 (separation)
	6602 (heuristics)

HiGHS printout notice

- The HiGHS print-out is slightly more complicated because HiGHS employs re-starts (deletes the tree)
- Again the main thing to consider is the Gap, BestBound and BestSol columns
- Afterwards a report is printed out. Notice that for this problem, HiGHS checks many fewer nodes than Gurobi (only 124) ...

Number of active nodes graph

For harder MIP problems the development of the number of active nodes can be illustrated:



Unfortunately, it is probably impossible to create a time-bar

How can the gap be reduced ?

- Reduce size of Big-M formulations
- Check different models (if there are several different correct models, choose the one with the lowest gap ...)
- Add cuts (difficult and not discussed here)

Big-M

What do I mean by "reduce big-M" ?

Remember the Microbrewery 2 assignment, where the brewing capacity was 120. Which of the two constraints below is best (we assume that there already is a domain constraint limit on the x variables):

- Small "big-M":

```
@variable(microbrewery2, 0 <= x[b=1:B,m=1:M] <= 120)
@constraint(microbrewery2, [b=1:B,m=1:M],
x[b,m] <= 120*q[b,m])
```

- Big "big-M"

```
@variable(microbrewery2, 0 <= x[b=1:B,m=1:M] <= 120)
@constraint(microbrewery2, [b=1:B,m=1:M],
x[b,m] <= 999*q[b,m])
```

Overall rule: Use the smallest big-M value as possible.

Model differences

Recall the TSP problem:

- Lecture 3, we looked at the subtour-elimination constraints, but there are many: $2^{(N-1)}$
- In assignment 6.16 (Tuesday 14/1) you will solve the TSP with the Miller–Tucker–Zemlin constraints. These are easier to implement and there are only N^2

The Subtour limitation constraints are better because the GAP, i.e. the difference between the objective value of optimal solution to the LP relaxation and the optimal IP value is much smaller.

Model Tips

Here are some suggestions for good working habits to make modelling easier:

- Define sets (e.g. $D = \text{length}(\text{depots})$)
- What is a variable and what is its meaning (and unit)? Make that absolutely clear to yourself and write it in the comments (and make the explanation in terms of the sets)
- Write comments in general

Model Tips

- Be careful about summing over several different variables. Example:

```
@objective(m, Min, sum(x[i,j] + y[i,k] + z[i,k]
                        for i in 1:2, j in 1:4, k in 1:6) )
```

gives the following objective function:

```
Min 4 x[1,1] + 3 y[1,1] + 3 z[1,1] + 3 y[1,2] + 3 z[1,2] + 3 y[1,3] + 3 z[1,3] + 3 y[1,4] + 3 z[1,4] + 4 x[1,2]
    + 4 x[1,3] + 4 x[2,1] + 3 y[2,1] + 3 z[2,1] + 3 y[2,2] + 3 z[2,2] + 3 y[2,3] + 3 z[2,3] + 3 y[2,4]
    + 3 z[2,4] + 4 x[2,2] + 4 x[2,3]
```

- You probably meant:

```
@objective(m, Min, sum(x[i,j] for i in 1:2, j in 1:3)
                      + sum(+ y[i,k] + z[i,k] for i in 1:2, k in 1:4) )
```

which gives the following objective function:

```
Min x[1,1] + x[1,2] + x[1,3] + x[2,1] + x[2,2] + x[2,3]
    + y[1,1] + z[1,1] + y[1,2] + z[1,2] + y[1,3] + z[1,3] + y[1,4] + z[1,4] + y[2,1] + z[2,1] + y[2,2] + z[2,2]
    + y[2,3] + z[2,3] + y[2,4] + z[2,4]
```


Model Tips II

- Write the model on paper first. Once you are happy with the model, implement it in Julia.
- Balance constraints:
 - Carefully make sure: What goes in must come out
 - Boundary conditions

Think back on the past days: How much time did you spend on debugging?

Think about what you have already learned !

How would you, a week ago, have solved

- Microbrewery 2?
- Factory Planning 2?

Some of the assignments you will face this week (and next week) will be significantly more realistic ...

DTU



Machine Learning for Mathematical Programming

Lecture 5

Why do you study to become an engineer ?

Besides having fun and getting a well paid job, we hope you want to learn something

Competence and insight

Here at DTU we want to educate **competent** and **insightful** engineers:

- Competent: You can **do** stuff, like e.g. formulating and implementing mathematical programming models
- Insightful: You **understand** the stuff your are making

Can Machine Learning do Mathematical programming

Yes, at least to some degree. In 2023 there was a competition, called Natural Language for Optimization (NL4Opt) competition which was held by Huawei Technologies, Canada in collaboration with the University of British Columbia, Canada. Since then a number of groups have worked on it.

A survey from 2025 "A Survey of Optimization Modeling Meets LLMs: Progress and Future Directions", Xiao et. al, IJCAI-25.

Can Machine Learning do Mathematical programming

Methods	NL4Opt	IndustryOR	EasyLP	ComplexLP	NLP4LP	ReSocratic	ComplexOR
Standard	61.2%	38.1%	70.3%	57.7%	73.6%	48.4%	42.9%
CoT	62.2%	40.5%	49.5%	42.3%	74.7%	43.6%	39.2%
Chain-of-Experts	66.7%	31.2%	94.4%	50.6%	87.4%	71.2%	57.1%
CAFA	68.1%	41.1%	71.2%	44.5%	50.0%	40.1%	46.4%
ORLM-LLaMA-3 8B	73.8%	42.9%	90.4%	59.5%	76.4%	61.8%	50.0%

But can this be improved in the next few years ?

ML better at modelling ?

Actually, it is hard to train AI to do mathematical programming modeling, because of missing data !

But if AI suddenly becomes VERY good at modeling ?

Then why should you do the modeling ?

Because:

- Maybe ML is wrong ... and you are responsible ...
- In the future you will be the quality-warranty of the model

An example: When we get self-driving-cars, you still have to learn to drive a car ...

Quote: Adam DeJans Jr., Gurobi

If you're an operations researcher who's mad that LLMs can now formulate basic linear programs, you've been coasting on syntax for way too long.

The optimization community is having a full identity crisis right now. AI can write LP formulations, handle the mathematical notation, and solve straightforward problems. And instead of adapting, some folks are acting like their entire profession just got invalidated.

If your primary skill set is "I know how to write constraints in the correct mathematical format," you weren't really doing operations research. You were being a human compiler.

Quote: Adam DeJans Jr., Gurobi

Operations Research has "Operations" in the name for a reason. The job was never about LaTeX proficiency or memorizing simplex algorithms. It was about:

- Understanding complex business operations well enough to know what needs optimizing
- Recognizing which messy real-world problems can even BE formulated mathematically
- Translating ambiguous business constraints into rigorous models
- Knowing when your mathematically optimal solution will fail in practice
- Actually getting stakeholders to implement your recommendations

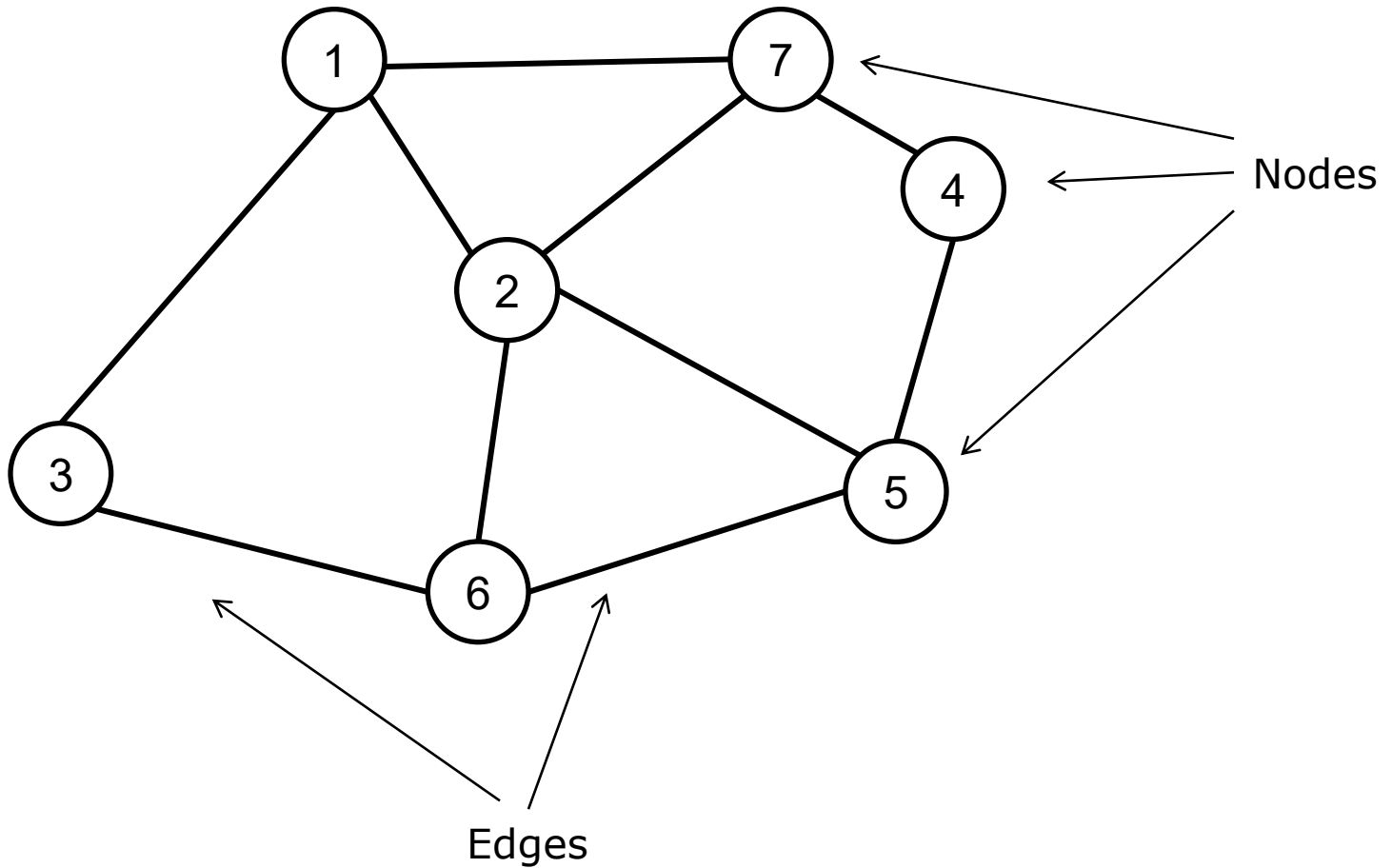
The Exam ...

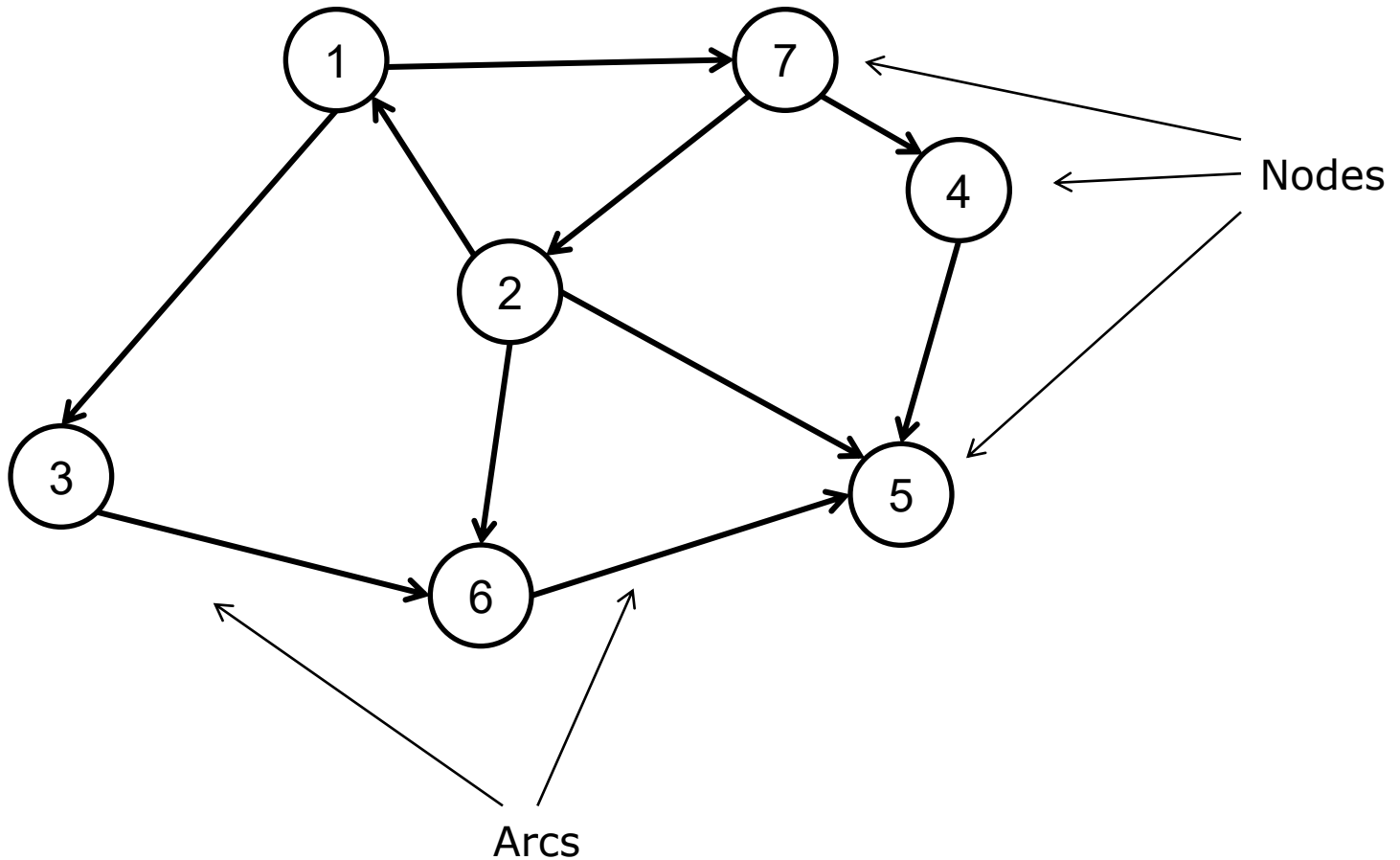
At the exam we will not allow the use of LLM's. And there will not be internet access. We expect you to be able to answer Multiple Choice questions **and** formulate and implement Mathematical Programming Models.

42112 – Lecture 6

Network models

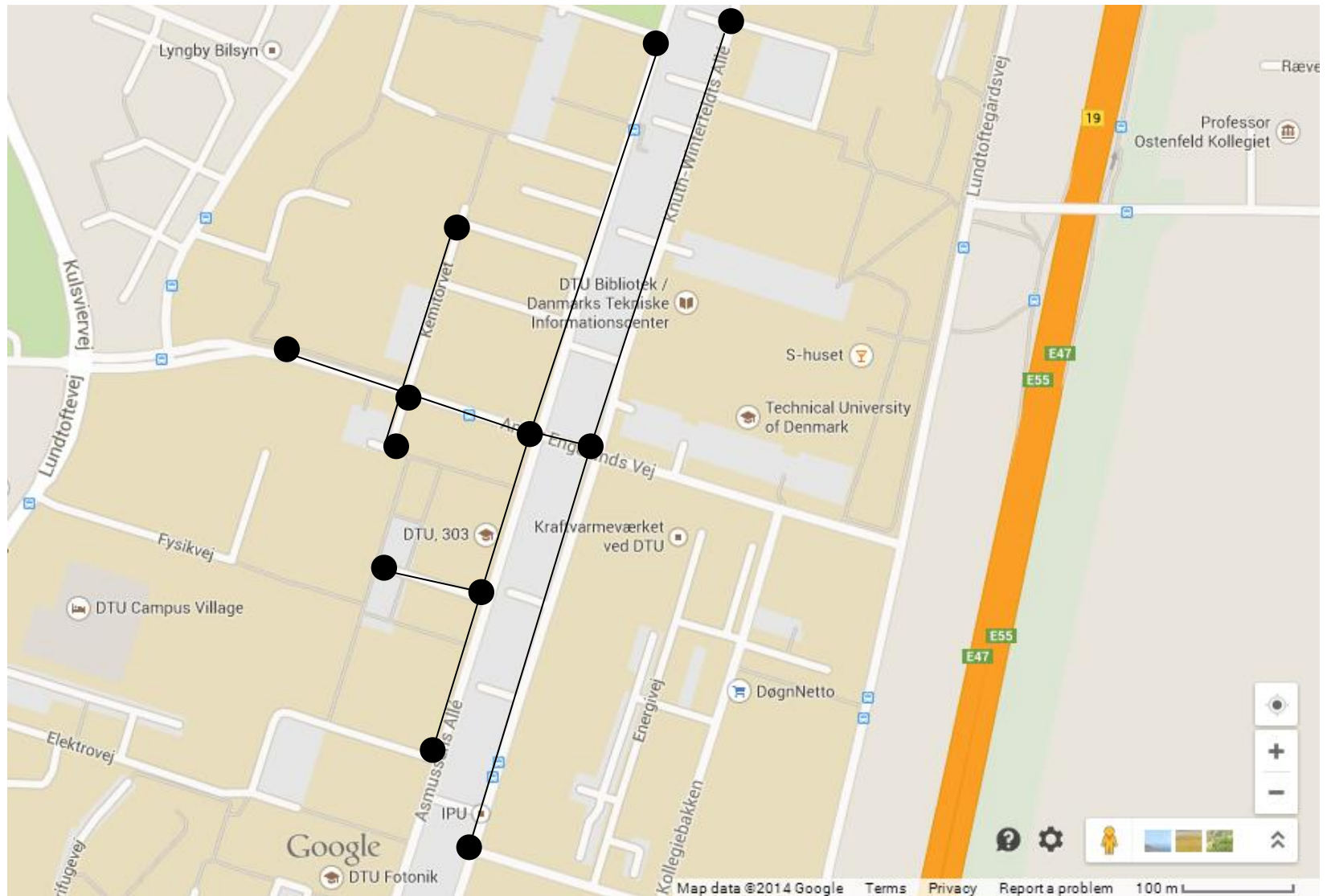
- Graphs / networks
- Shortest path problem
- Minimum cost flow problem
- Multi-commodity flow problem
- Network design (fixed charge multi-commodity flow problem)





DTU

Examples of graphs- Road networks



DTU

Examples of graphs - Sewer systems

- Can we transport the water away quickly enough?

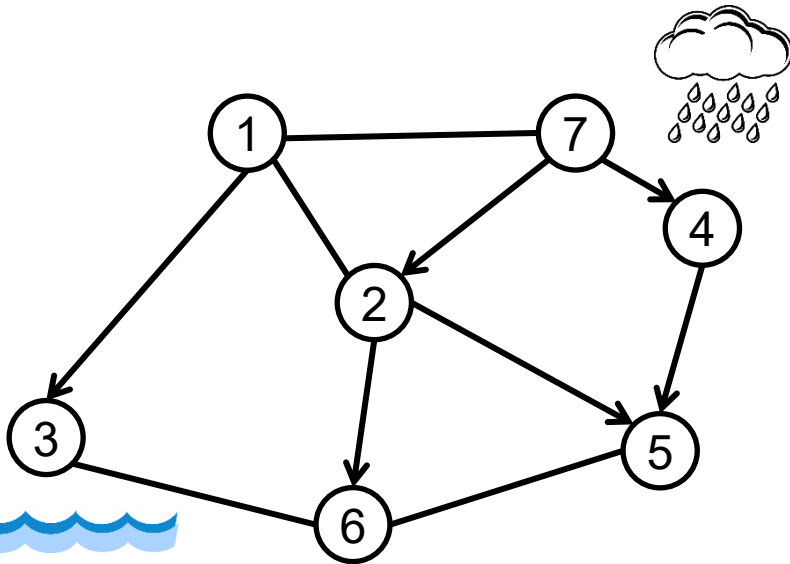


Foto: Lisa Risager

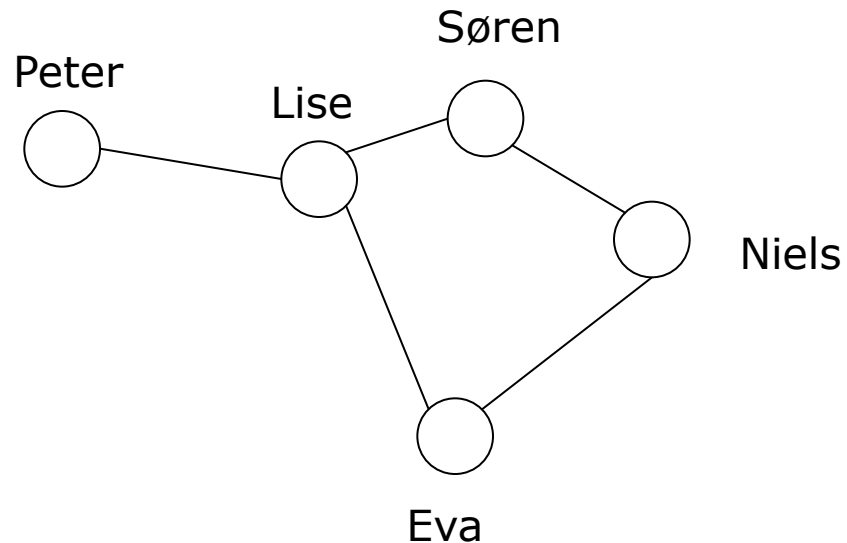
License: Attribution-ShareAlike 2.0 Generic

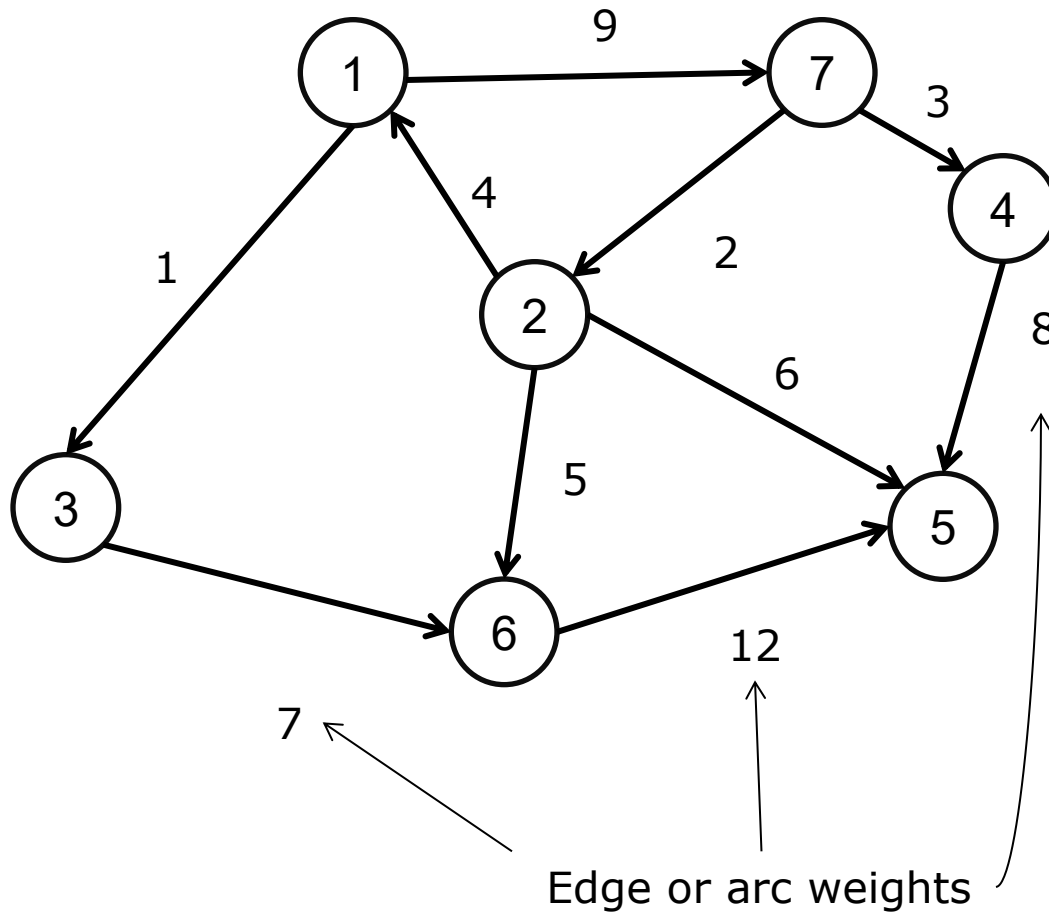
Istedgade July 2011

Examples of graphs– social networks



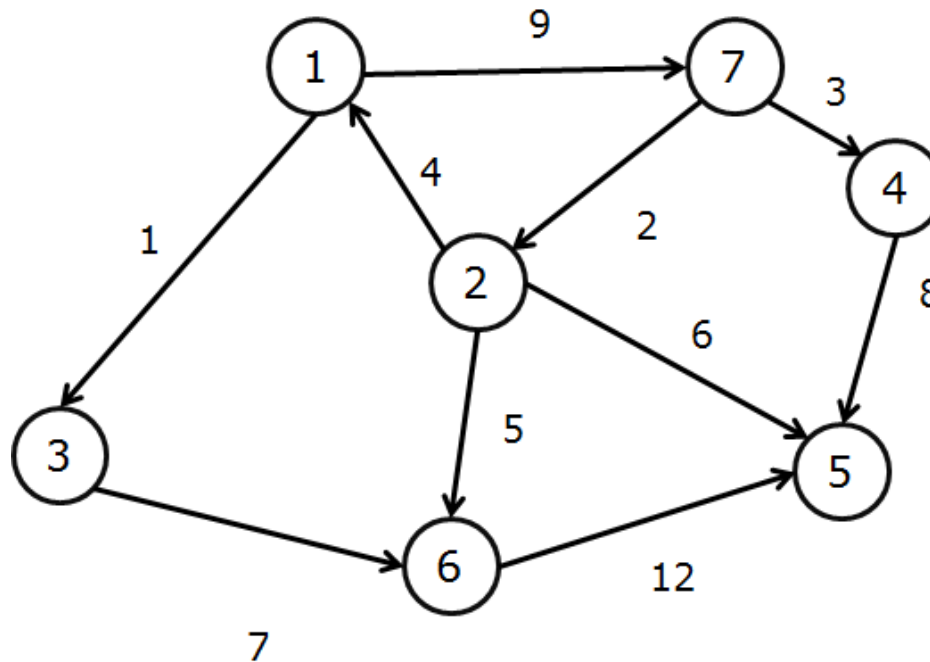
- Who knows who?
- Who follows who?



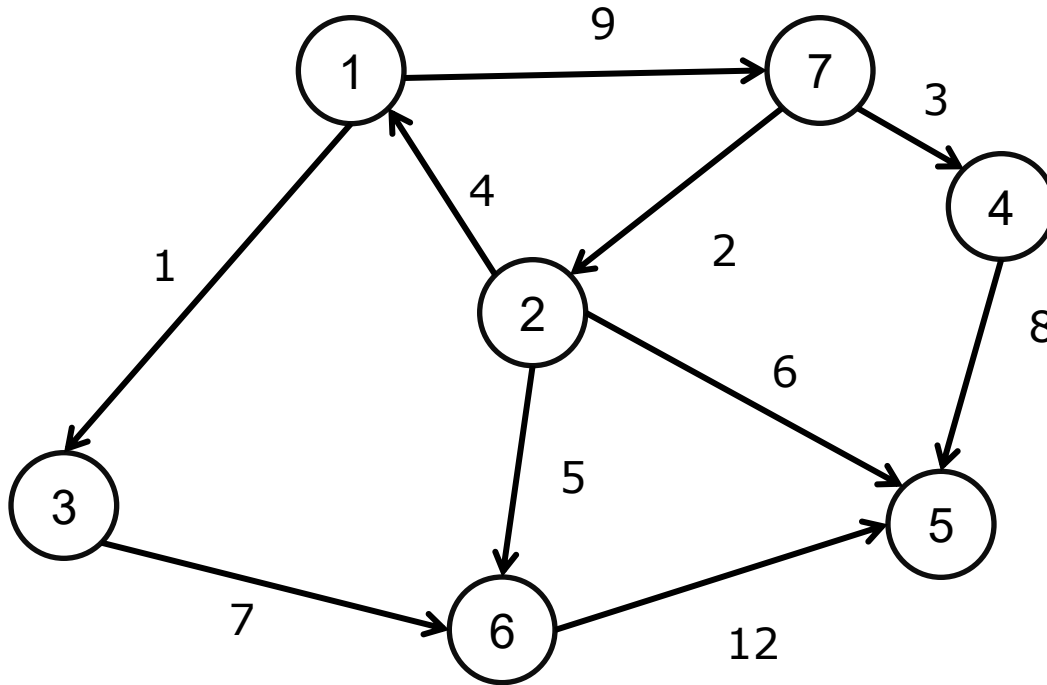


(Could represent arc capacity, cost, distance, etc.)

Nodes	Arcs	Flow
Intersections	Roads	Vehicles
Switching points	Wires/channels	Messages/Data
Pumping stations	Pipes	Fluids

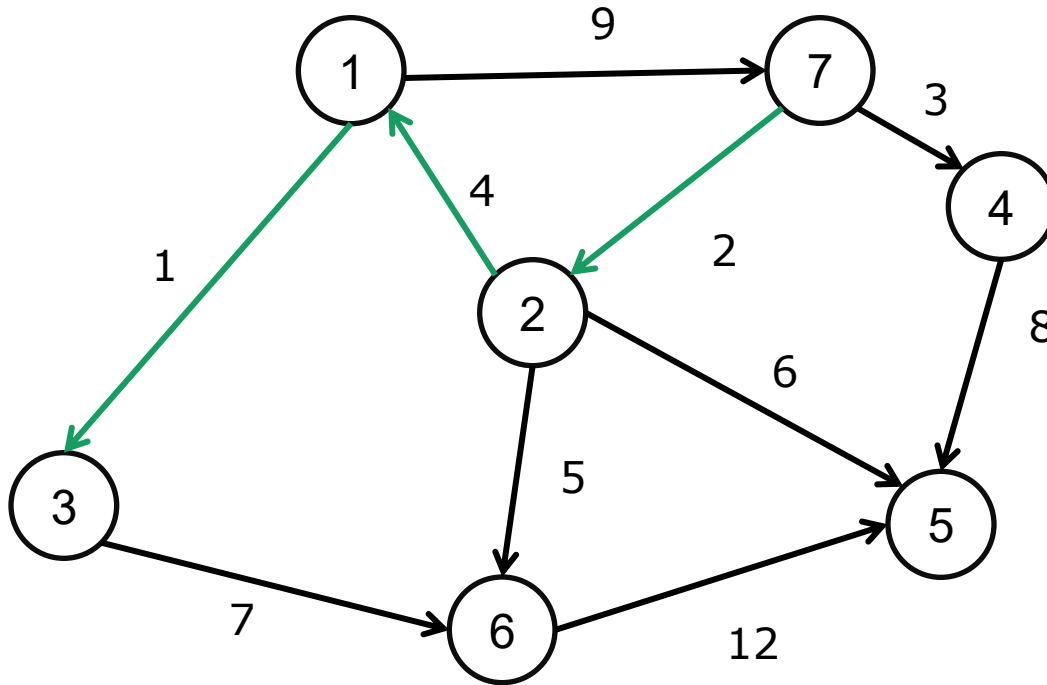


Model that finds the shortest/cheapest path from 7 to 3?



- Strategy: Send one unit from 7 to 3.
- Objective: for each unit we send through an arc we pay the cost of the associated arc
- One unit should leave node 7
- One unit should enter node 3
- For all other nodes: The number of units that goes in must be equal to the number units going out.

Model that finds the shortest/cheapest path from 7 to 3?



- Strategy: Send one unit from 7 to 3.
- Objective: for each unit we send through an arc we pay the cost of the associated arc
- One unit should leave node 7
- One unit should enter node 3
- For all other nodes: The number of units that goes in must be equal to the number units going out.

- c_{ij} : cost of arc (i, j)
- s, t : source and destination for the shortest path computation
- N : number of nodes.
- A : set of arcs that can be used

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

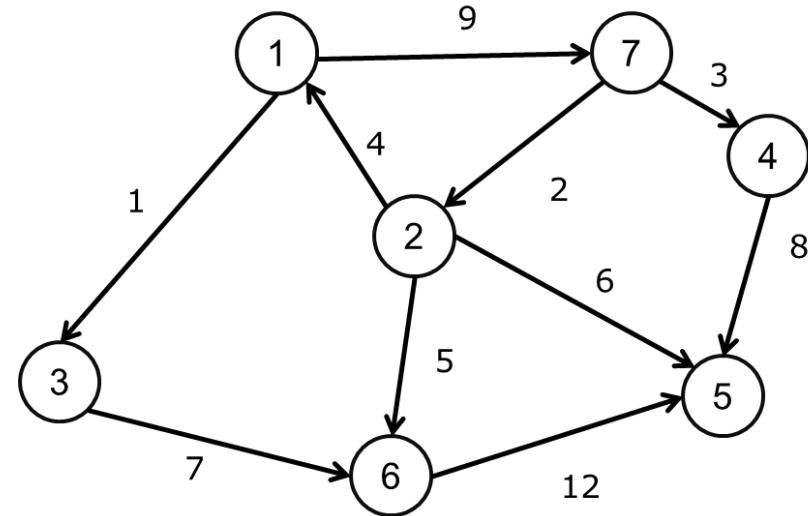
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \in \{0, 1\}$$



- Strategy: Send one unit from 7 to 3.
- Objective: for each unit we send through an arc we pay the cost of the associated arc
- One unit should leave node 7
- One unit should enter node 3
- For all other nodes: The number of units that goes in must be equal to the number units going out.

- c_{ij} : cost of arc (i, j)
- s, t : source and destination for the shortest path computation
- N : number of nodes.
- A : set of arcs that can be used

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

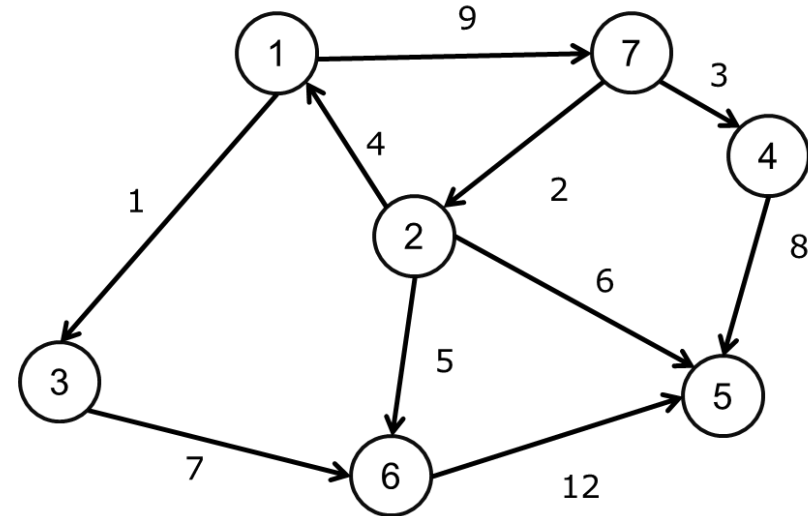
$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$

Integer solution is guaranteed (theory of totally unimodular matrices)



Implementation version 1: fixing variables

```
using JuMP
using HiGHS
```

```
N = 7 # Number of nodes
S = 7 # Source
T = 3 # Target
```

```
# -----
# Cost matrix c[i,j]
# -----
```

```
c = [
    0 0 1 0 0 0 9
    4 0 0 0 6 5 0
    0 0 0 0 0 7 0
    0 0 0 0 8 0 0
    0 0 0 0 0 0 0
    0 0 0 0 12 0 0
    0 2 0 3 0 0 0
]
```

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

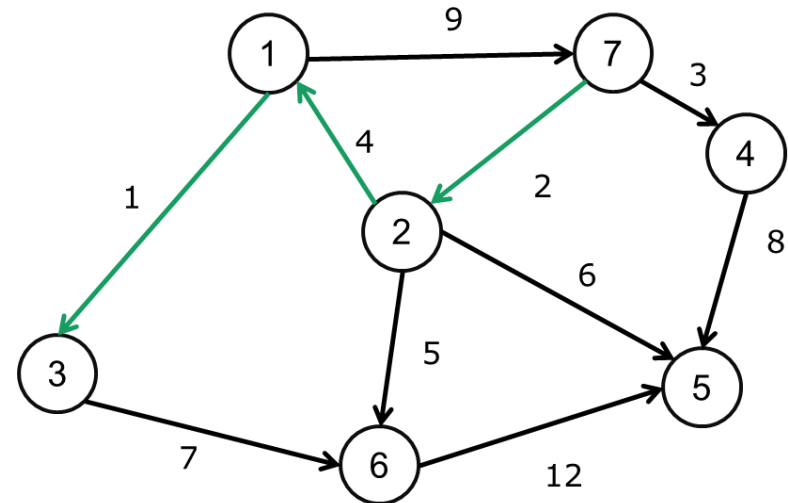
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$



Implementation version 1: fixing variables

```
# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

# Flow variables on ALL pairs
@variable(model, x[1:N, 1:N] >= 0)

# Fix infeasible arcs to zero (where c[i,j] = 0)
for i in 1:N, j in 1:N
    if c[i,j] == 0
        fix(x[i,j], 0.0; force = true)
    end
end
```

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

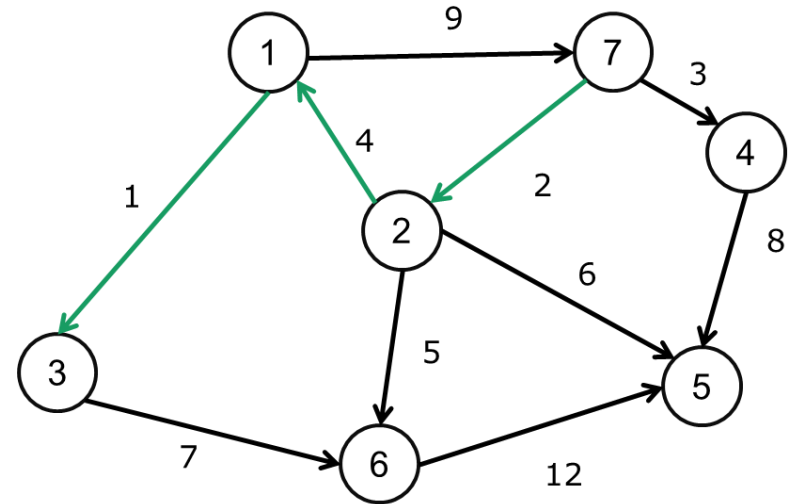
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$



Implementation version 1: fixing variables

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$

```
# Objective: minimize total cost
@objective(model, Min,
    sum(c[i,j] * x[i,j] for i in 1:N, j in 1:N)
)

# -----
# Flow conservation constraints
# -----

# Source: outflow - inflow = 1
@constraint(model, sum(x[S,j] for j in 1:N) - sum(x[i,S] for i in 1:N) == 1)

# Destination: inflow - outflow = 1
@constraint(model, sum(x[i,T] for i in 1:N) - sum(x[T,j] for j in 1:N) == 1)

# Intermediate nodes: inflow = outflow
for v in 1:N
    if v != S && v != T
        @constraint(model,
            sum(x[i,v] for i in 1:N) == sum(x[v,j] for j in 1:N)
        )
    end
end
```

Implementation version 1: fixing variables

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$\begin{aligned} x_{ij} &= 0 \\ x_{ij} &\geq 0 \end{aligned} \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

```
# -----
# Solve
# -----
optimize!(model)
if termination_status(model) == MOI.OPTIMAL
    println("Termination status: ", termination_status(model))
    println("Optimal objective  ", objective_value(model))

    println("\nChosen arcs:")
    for i in 1:N, j in 1:N
        if value(x[i,j]) > 1e-6
            println(" x[$i,$j] = $(value(x[i,j]))")
        end
    end
else
    println("Model not optimal (maybe infeasible).")
end
```

Implementation version 1: fixing variables

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$

Termination status: OPTIMAL

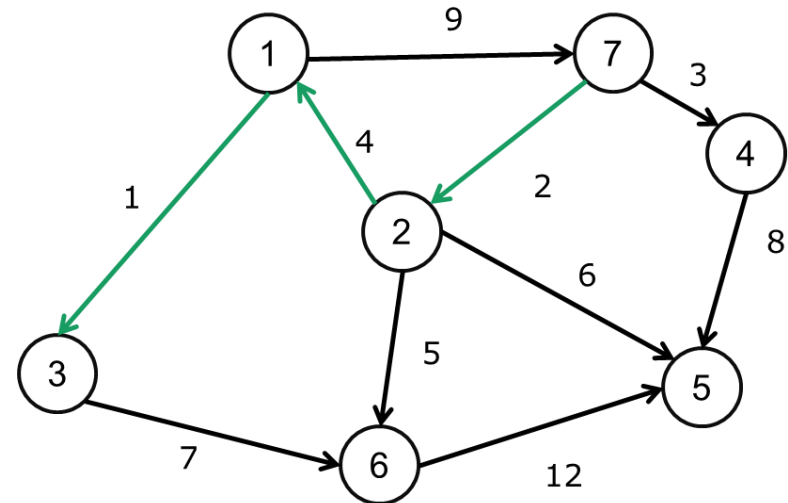
Optimal objective 7.0

Chosen arcs:

$$x[1,3] = 1.0$$

$$x[2,1] = 1.0$$

$$x[7,2] = 1.0$$



Implementation version 2: create only needed vars.

using JuMP, HiGHS

N = 7 # Number of nodes

S = 7

T = 3

Directed arcs

```
arcs = [
    (1,3), (1,7),
    (2,1), (2,5), (2,6),
    (3,6),
    (4,5),
    (6,5),
    (7,2), (7,4)
]
```

]

Arc costs

```
c = Dict{Tuple{Int,Int}, Int}()
```

```
c[(1,3)] = 1
```

```
c[(1,7)] = 9
```

```
c[(2,1)] = 4
```

```
c[(2,5)] = 6
```

```
c[(2,6)] = 5
```

```
c[(3,6)] = 7
```

```
c[(4,5)] = 8
```

```
c[(6,5)] = 12
```

```
c[(7,2)] = 2
```

```
c[(7,4)] = 3
```

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0$$

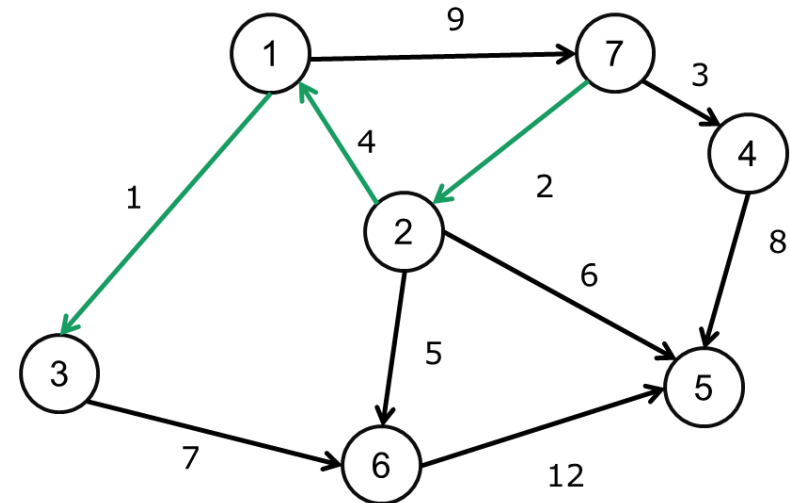
$$\forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0$$

$$\forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$



Implementation version 2: create only needed vars.

```
# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, x[a in arcs] >= 0)

@objective(model, Min, sum(c[a] * x[a] for a in arcs))

# 1) Source (Copenhagen): outflow - inflow = 1
@constraint(model,
    sum(x[a] for a in arcs if a[1] == S) -
    sum(x[a] for a in arcs if a[2] == S) == 1
)

# 2) Intermediate nodes: inflow = outflow
@constraint(model, [k in 1:N; k != S && k != T],
    sum(x[a] for a in arcs if a[2] == k) ==
    sum(x[a] for a in arcs if a[1] == k)
)

# 3) Destination (Sydney): inflow - outflow = 1
@constraint(model,
    sum(x[a] for a in arcs if a[2] == T) -
    sum(x[a] for a in arcs if a[1] == T) == 1
)
```

s.t.

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

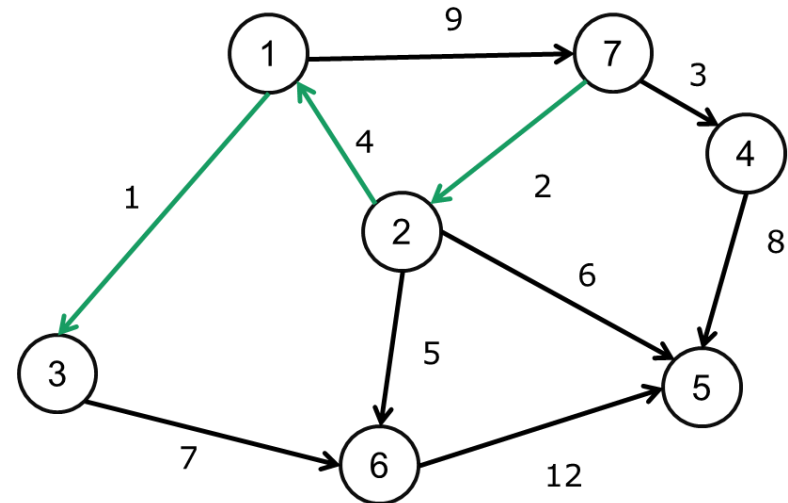
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$



Implementation version 2: create only needed vars.

```

optimize!(model)

println("Termination status: ", termination_status(model))

if termination_status(model) == MOI.OPTIMAL
    println("Objective value: ", objective_value(model))
    println("\nFlows on arcs:")
    for a in arcs
        if value(x[a]) > 1e-6
            println(" Flow on arc $a is ", value(x[a]))
        end
    end
else
    println("Not optimal (model maybe infeasible).")
end

```

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

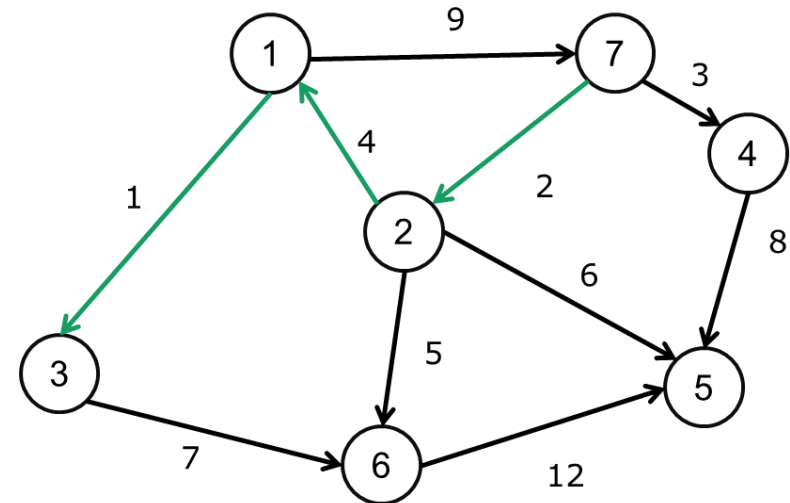
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$



Implementation version 2: create only needed vars.

Termination status: OPTIMAL

Objective value: 7.0

Flows on arcs:

Flow on arc (1, 3) is 1.0

Flow on arc (2, 1) is 1.0

Flow on arc (7, 2) is 1.0

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

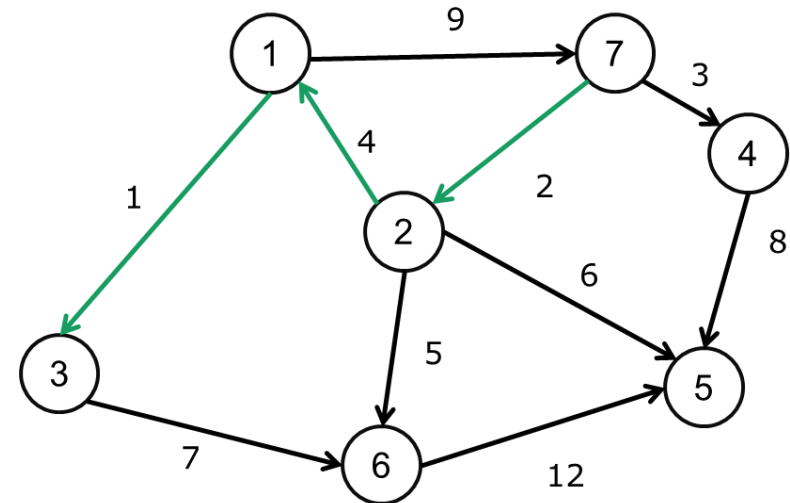
$$\sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} = 1$$

$$\sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} = 0 \quad \forall k = 1, \dots, N, k \neq s, k \neq t$$

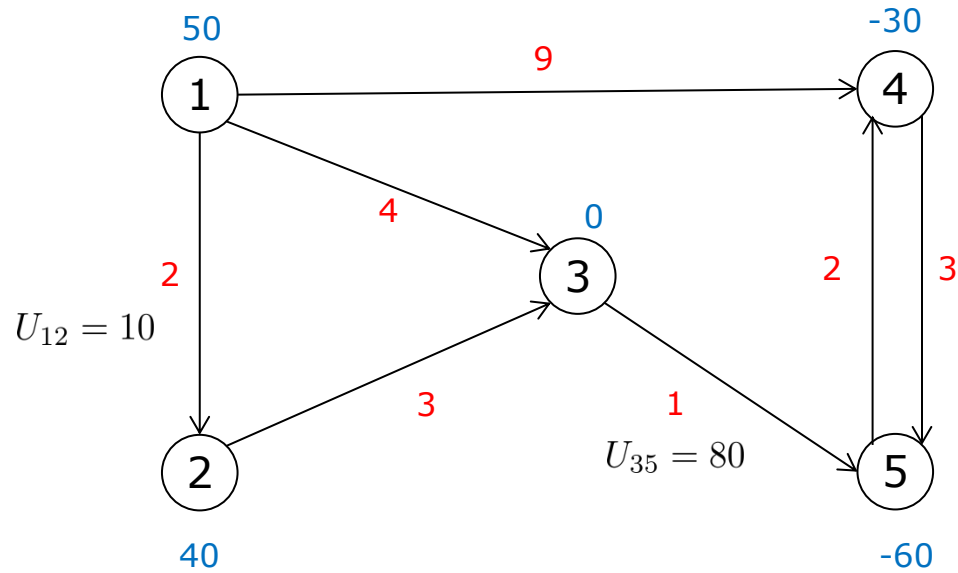
$$\sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} = -1$$

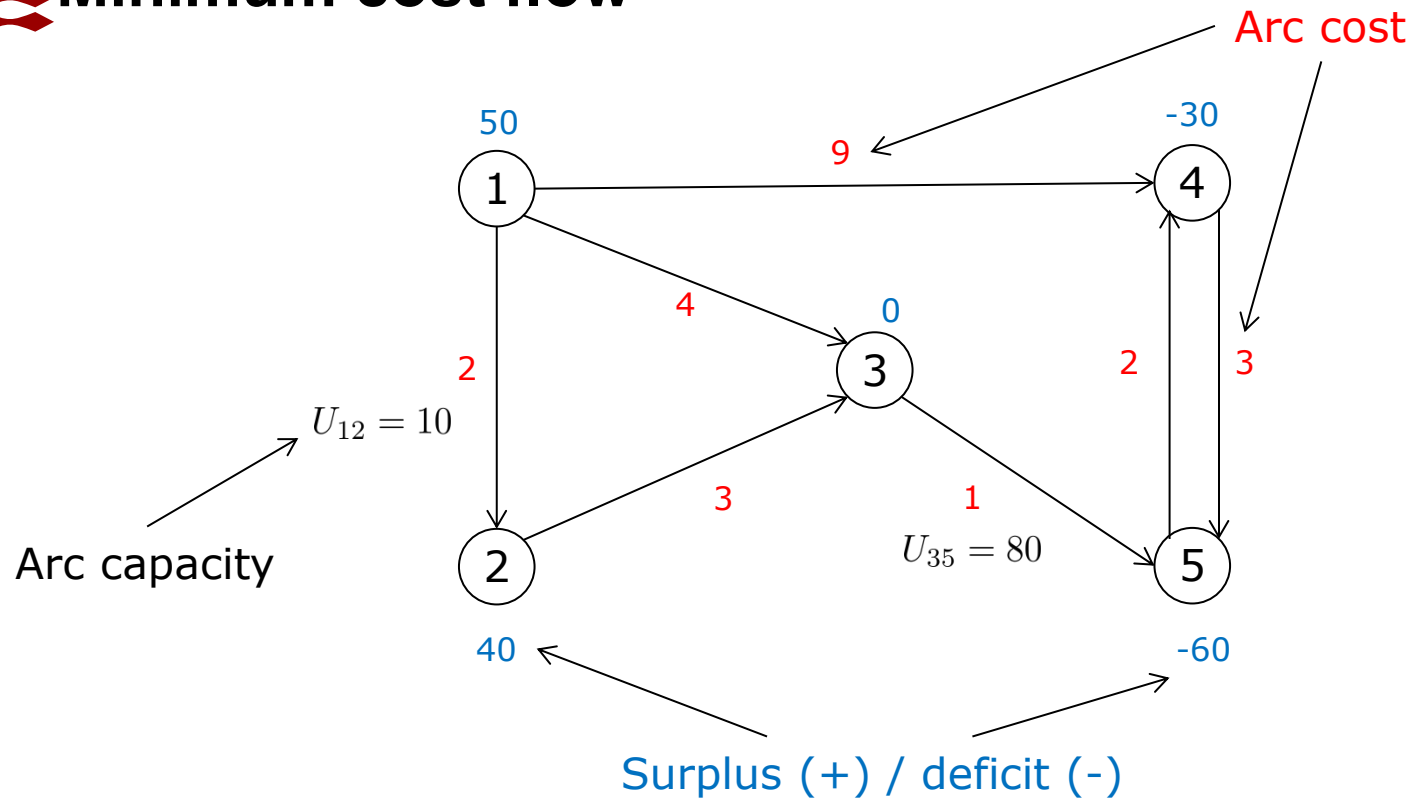
$$x_{ij} = 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A$$

$$x_{ij} \geq 0$$

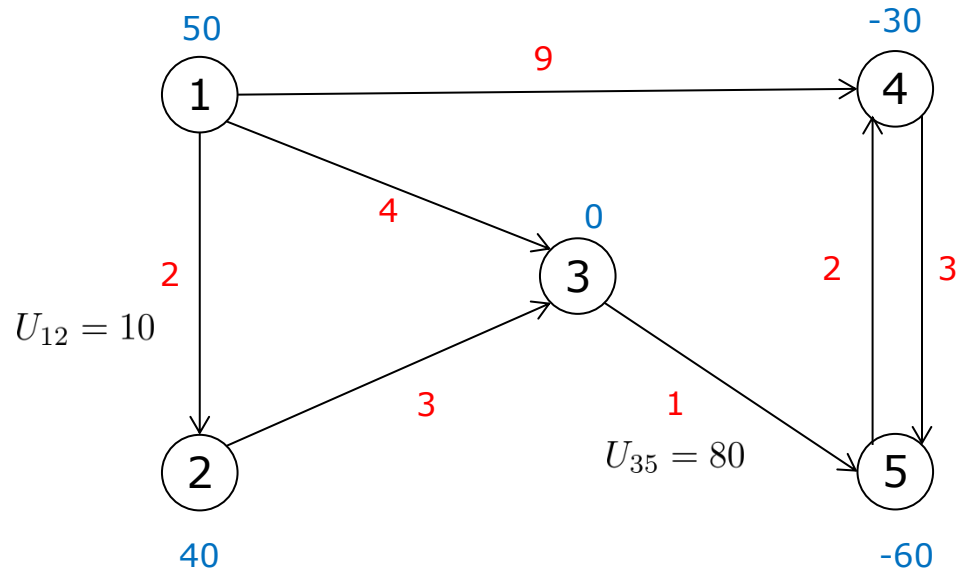


DTU Minimum cost flow



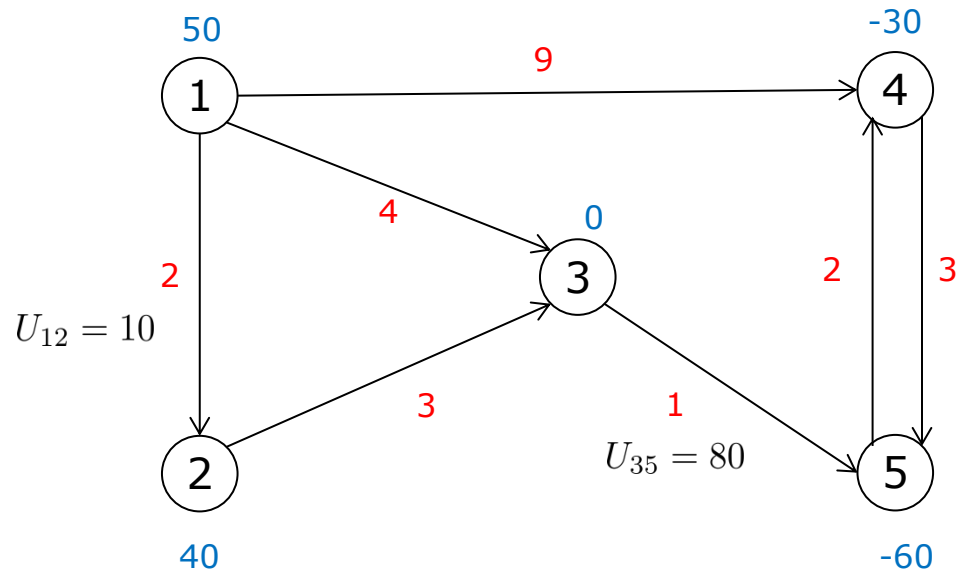


DTU Minimum cost flow



Decision variables:

x_{ij} : how many units we send through the arc from node i to node j .



$$\min 2x_{12} + 4x_{13} + 9x_{14} + 3x_{23} + 1x_{35} + 3x_{45} + 2x_{54}$$

subject to

$$x_{12} + x_{13} + x_{14} = 50$$

$$-x_{12} + x_{23} = 40$$

$$-x_{13} - x_{23} + x_{35} = 0$$

$$-x_{14} + x_{45} - x_{54} = -30$$

$$-x_{35} - x_{45} + x_{54} = -60$$

$$x_{12} \leq 10$$

$$x_{35} \leq 80$$

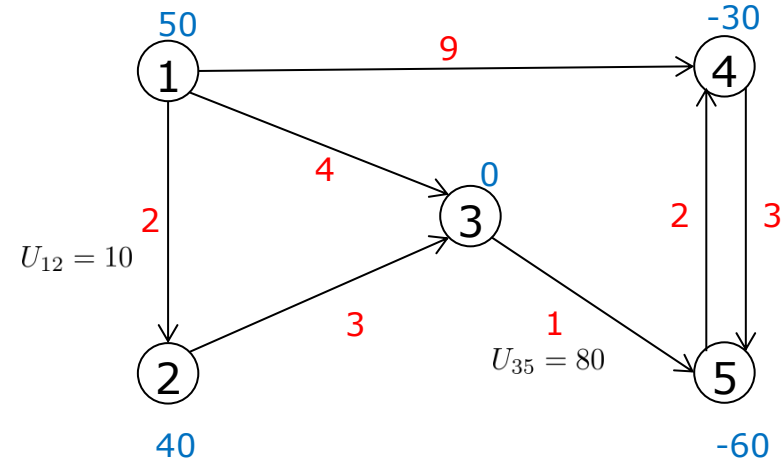
$$x_{12}, x_{13}, x_{14}, x_{23}, x_{35}, x_{45}, x_{54} \geq 0$$

Minimum cost flow : Generic/abstract model

- m : number of nodes in the network
- n : number of arcs in the network
- (i, j) the arc going from i to j
- c_{ij} : cost for sending one unit through arc (i, j)
- u_{ij} : capacity of arc (i, j)
- b_i : surplus / deficit at node i
- A : set of all arcs in the network

Decision variables:

- x_{ij} : The amount of goods to send through arc (i, j)



$$\min 2x_{12} + 4x_{13} + 9x_{14} + 3x_{23} + 1x_{35} + 3x_{45} + 2x_{54}$$

subject to

$$\begin{aligned} x_{12} + x_{13} + x_{14} &= 50 \\ -x_{12} &+ x_{23} &= 40 \\ &- x_{13} &- x_{23} + x_{35} &= 0 \\ &&- x_{14} &+ x_{45} - x_{54} &= -30 \\ &&&- x_{35} - x_{45} + x_{54} &= -60 \\ x_{12} &\leq 10 \\ &x_{35} &\leq 80 \\ x_{12}, x_{13}, x_{14}, x_{23}, x_{35}, x_{45}, x_{54} &\geq 0 \end{aligned}$$

Minimum cost flow : Generic/abstract model

- m : number of nodes in the network
- n : number of arcs in the network
- (i, j) the arc going from i to j
- c_{ij} : cost for sending one unit through arc (i, j)
- u_{ij} : capacity of arc (i, j)
- b_i =surplus / deficit at node i
- A : set of all arcs in the network

Decision variables:

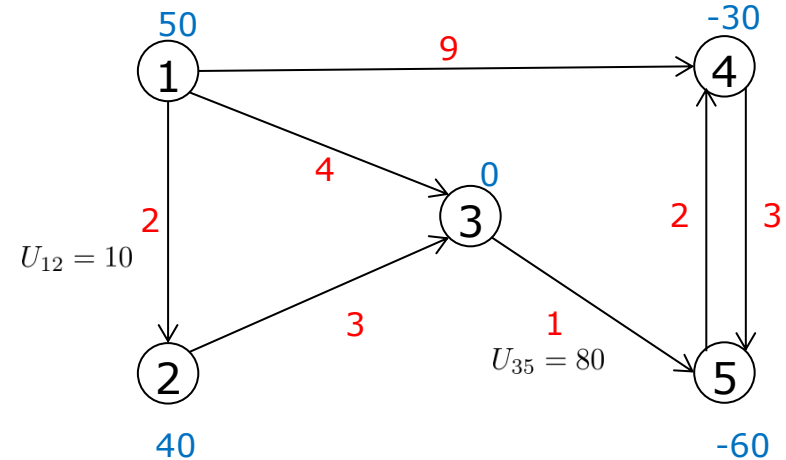
- x_{ij} : The amount of goods to send through arc (i, j)

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

subject to

$$\sum_{(i,j) \in A: i=k} x_{ij} - \sum_{(i,j) \in A: j=k} x_{ij} = b_k \quad \forall k = 1, \dots, m$$

$$0 \leq x_{ij} \leq u_{ij} \quad \forall (i, j) \in A$$



$$\min 2x_{12} + 4x_{13} + 9x_{14} + 3x_{23} + 1x_{35} + 3x_{45} + 2x_{54}$$

subject to

$$\begin{aligned} x_{12} + x_{13} + x_{14} &= 50 \\ -x_{12} &+ x_{23} &= 40 \\ &- x_{13} &- x_{23} + x_{35} &= 0 \\ &&- x_{14} &+ x_{45} - x_{54} &= -30 \\ &&&- x_{35} - x_{45} + x_{54} &= -60 \\ x_{12} &&&&&\leq 10 \\ &&&x_{35} &&\leq 80 \\ x_{12}, x_{13}, x_{14}, x_{23}, x_{35}, x_{45}, x_{54} &\geq 0 \end{aligned}$$

Minimum cost flow : Julia implementation

```

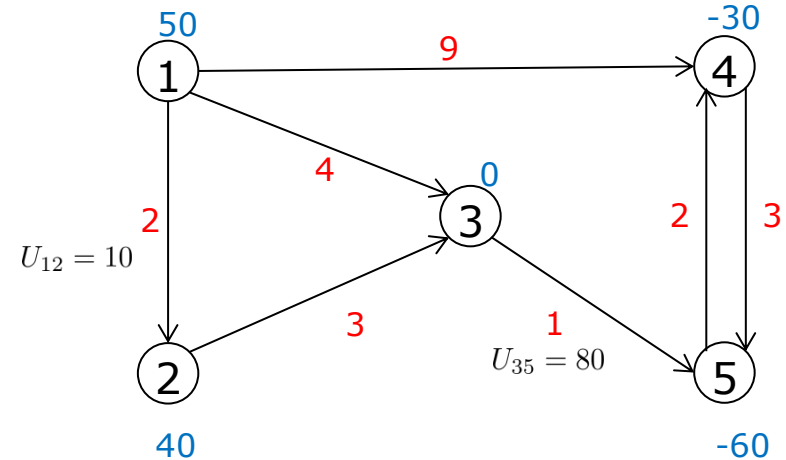
using JuMP, HiGHS

N = 5 # Number of nodes
# Arcs
arcs = [
    (1,2), (1,3), (1,4),
    (2,3),
    (3,5),
    (4,5),
    (5,4)
]

# -----
# Arc costs (hundreds of euros)
# -----
c = Dict{Tuple{Int,Int}, Int}()
c[(1,2)] = 2
c[(1,3)] = 4
c[(1,4)] = 9
c[(2,3)] = 3
c[(3,5)] = 1
c[(4,5)] = 3
c[(5,4)] = 2
# -----
# upper bounds
# -----
u = Dict{Tuple{Int,Int}, Float64}()
u[(1,2)] = 10.0
u[(1,3)] = Inf
u[(1,4)] = Inf
u[(2,3)] = Inf
u[(3,5)] = 80.0
u[(4,5)] = Inf
u[(5,4)] = Inf

demands = [50, 40, 0, -30, -60]

```



$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

subject to

$$\sum_{(i,j) \in A: i=k} x_{ij} - \sum_{(i,j) \in A: j=k} x_{ij} = b_k \quad \forall k = 1, \dots, m$$

$$0 \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A$$

Minimum cost flow : Julia implementation

```

# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, 0 <= x[a in arcs] <= u[a])

@objective(model, Min, sum(c[a] * x[a] for a in arcs))

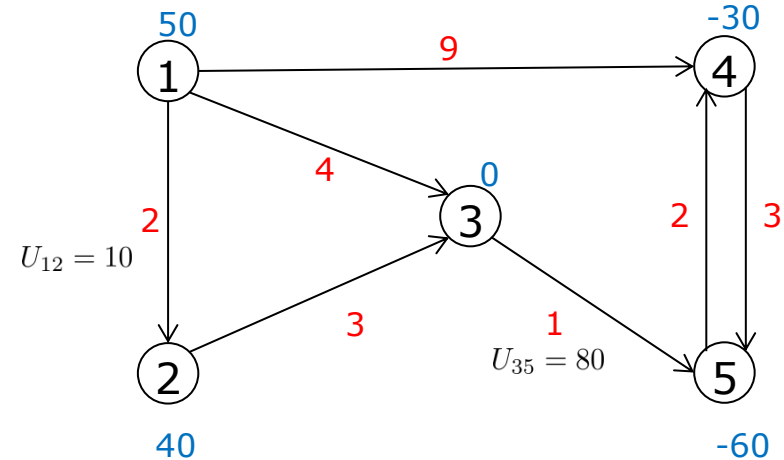
@constraint(model, [k in 1:N],
    sum(x[a] for a in arcs if a[1] == k)
    - sum(x[a] for a in arcs if a[2] == k) == demands[k]
)

optimize!(model)

println("Termination status: ", termination_status(model))

if termination_status(model) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(model))
    for a in arcs
        if JuMP.value.(x[a]) > 1e-6
            println("Flow on arc ($(a[1]), $(a[2])) is ",
                JuMP.value.(x[a]))
        end
    end
end
end

```



$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

subject to

$$\sum_{(i,j) \in A: i=k} x_{ij} - \sum_{(i,j) \in A: j=k} x_{ij} = b_k \quad \forall k = 1, \dots, m$$

$$0 \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A$$

Minimum cost flow : Julia implementation

Termination status: OPTIMAL

Objective value: 490.0

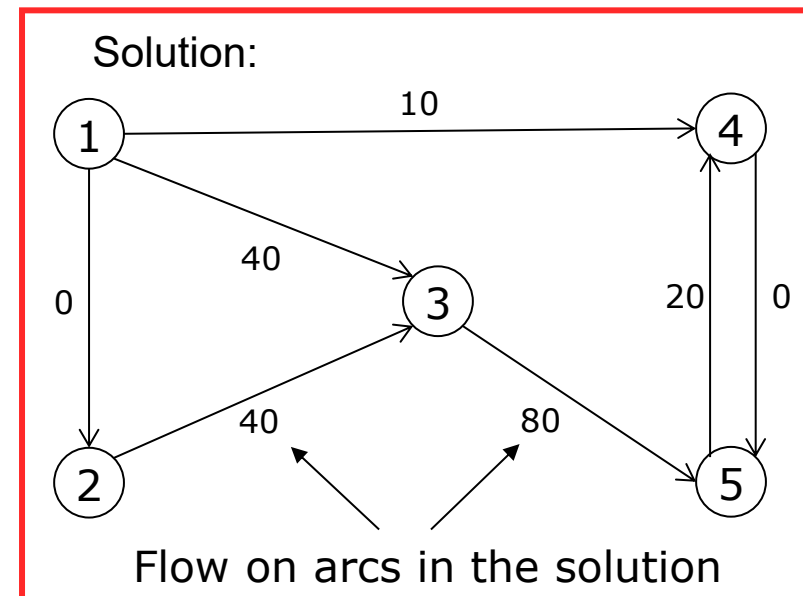
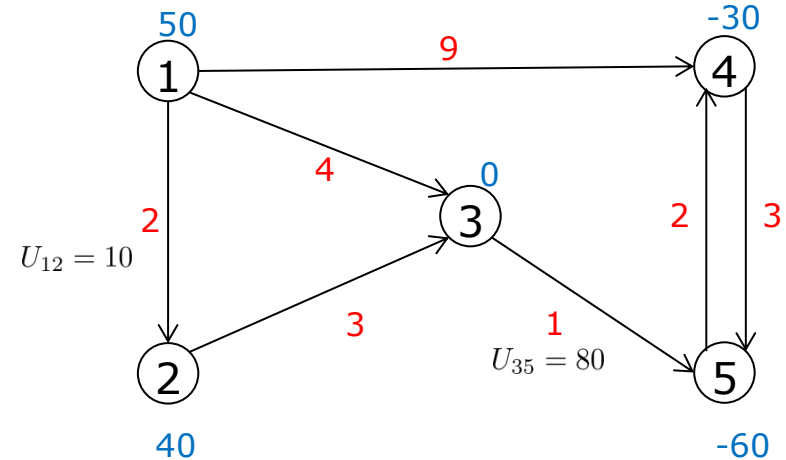
Flow on arc (1, 3) is 40.0

Flow on arc (1, 4) is 10.0

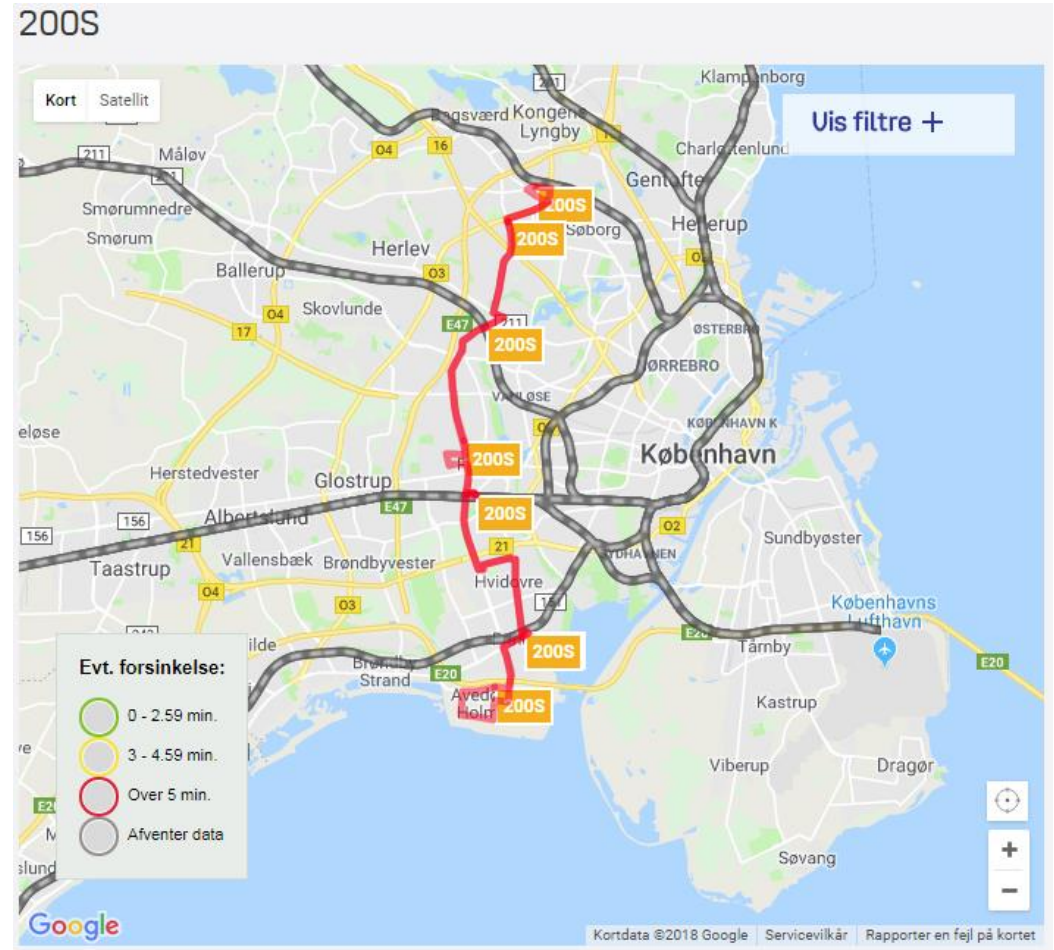
Flow on arc (2, 3) is 40.0

Flow on arc (3, 5) is 80.0

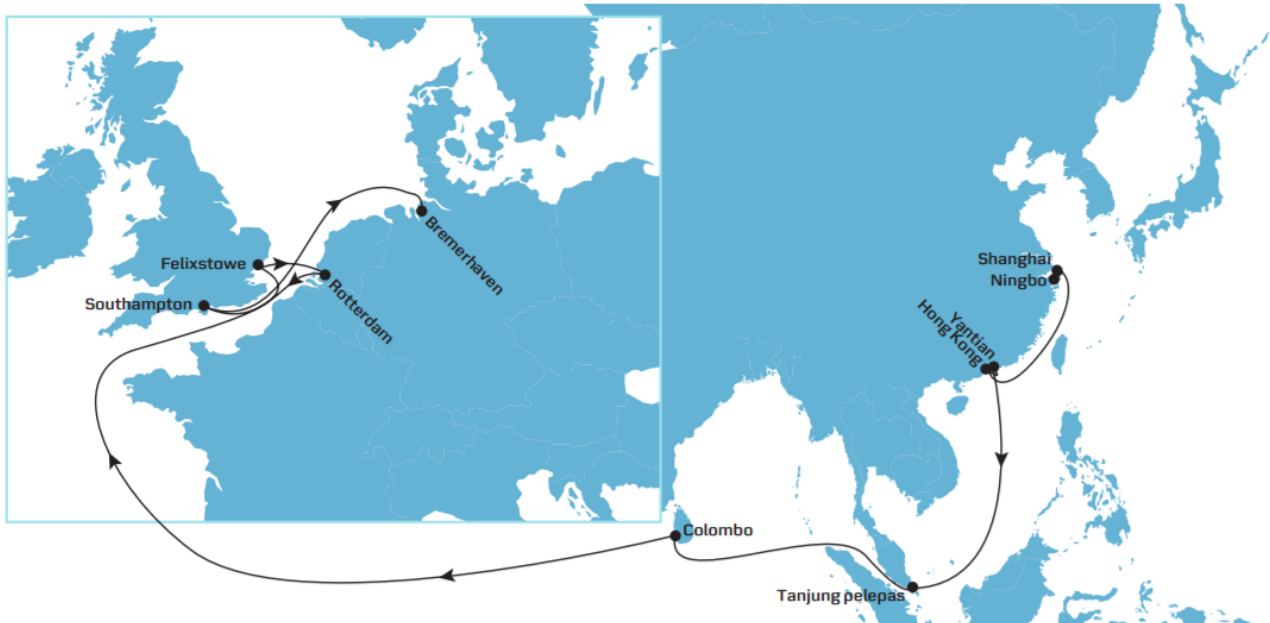
Flow on arc (5, 4) is 20.0



DTU Container ships... The buses of the sea

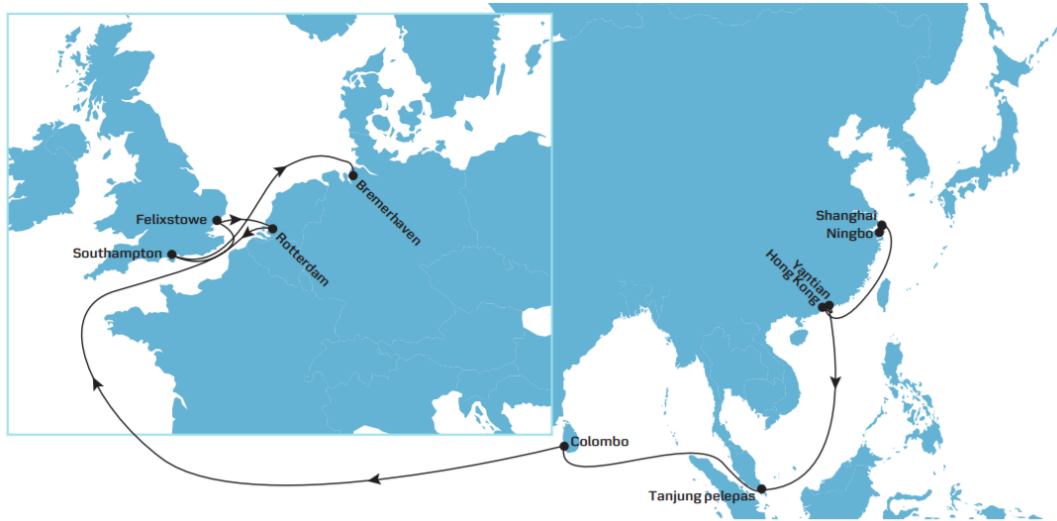


DTU Container ships... The buses of the sea

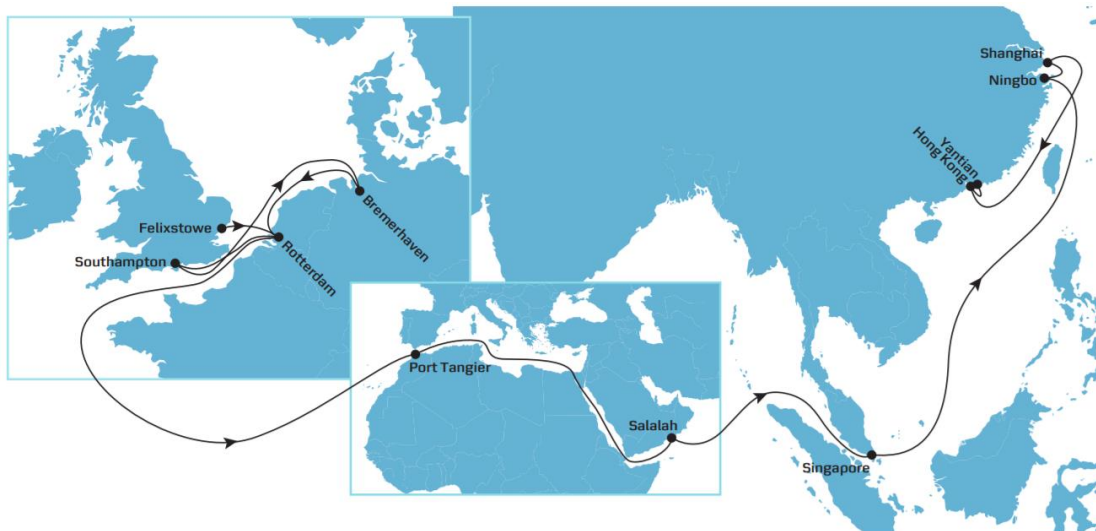


AE1

West bound



East bound



080 Boomerang

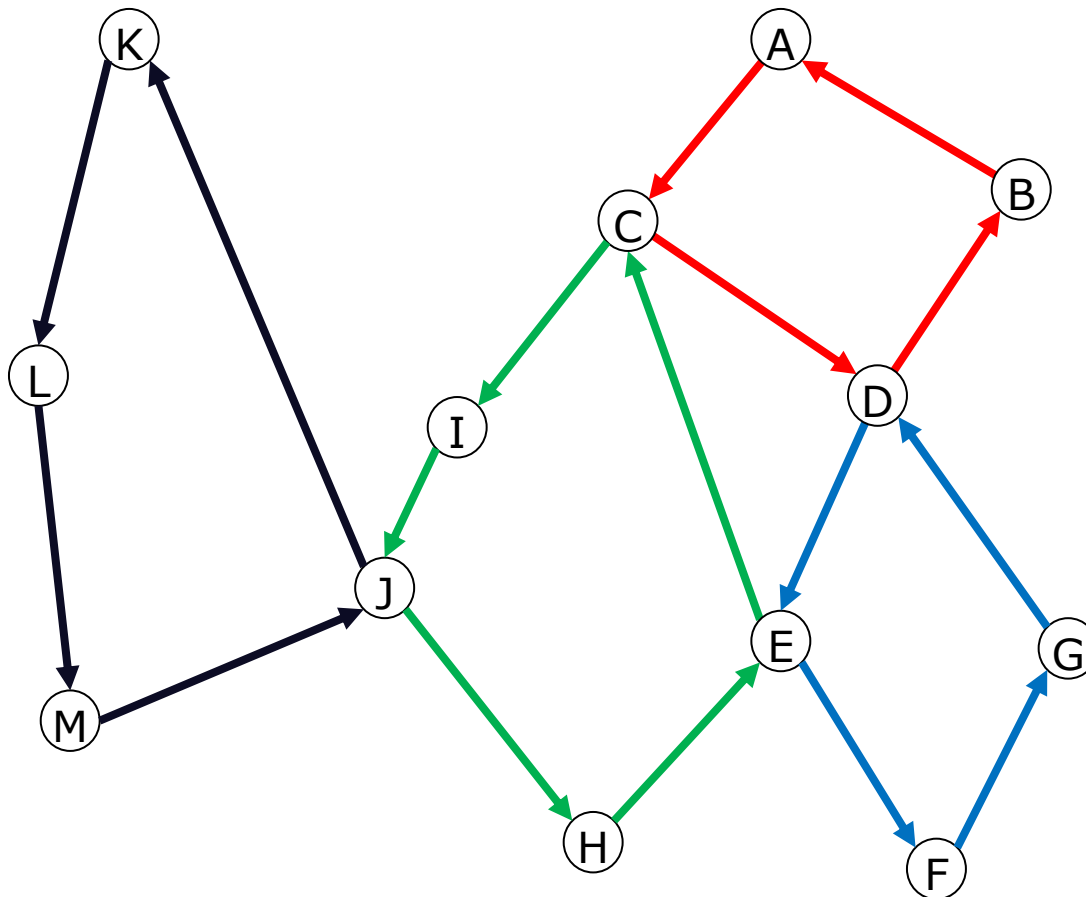


Need to ship a container from Melbourne to Bremerhaven?

- Load onto 080 in Melbourne
- Transship to AE1 west in Tanjung Pelepas
- Offload in Bremerhaven

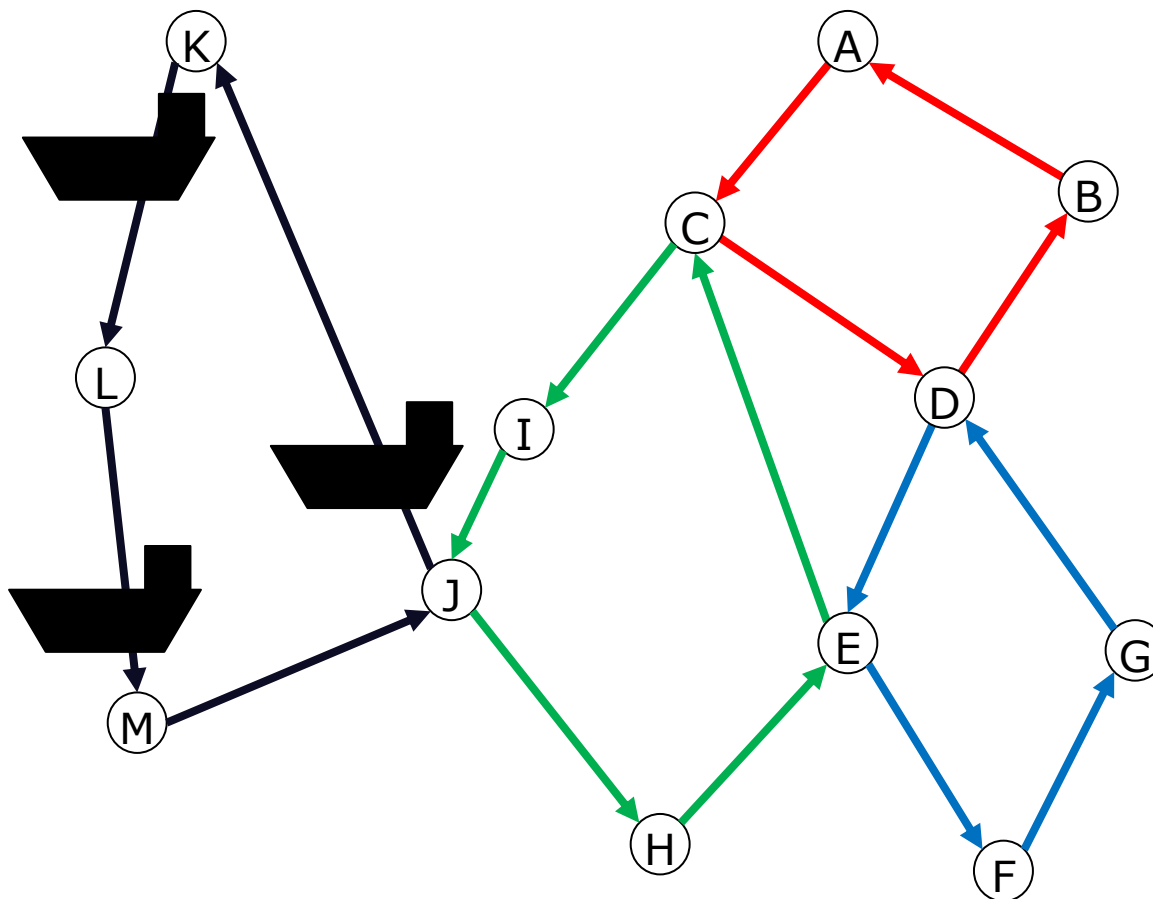
DTU "abstract" network

- Nodes = ports
- Colored cycles = "rotations"



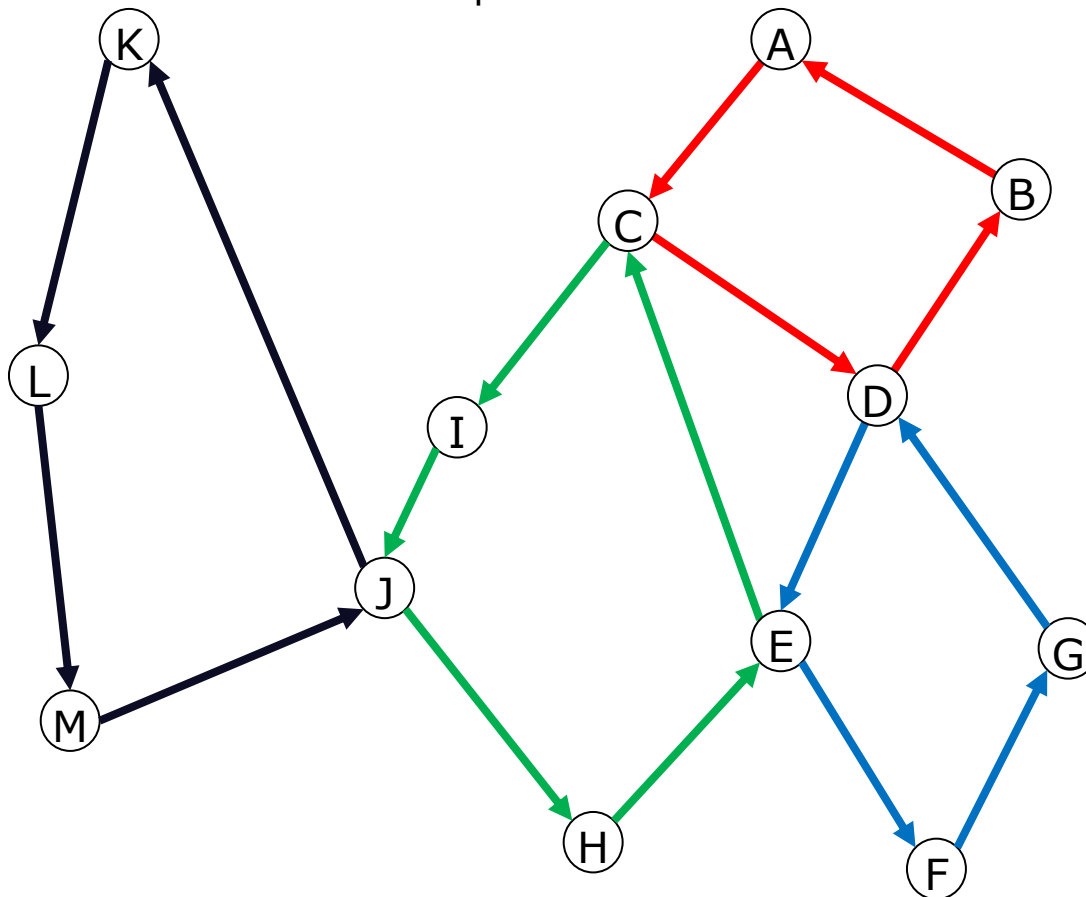
DTU "abstract" network

- Each rotation has a weekly visit frequency
- => if it takes three weeks to complete the dark blue rotation then we need three ships to operate it.



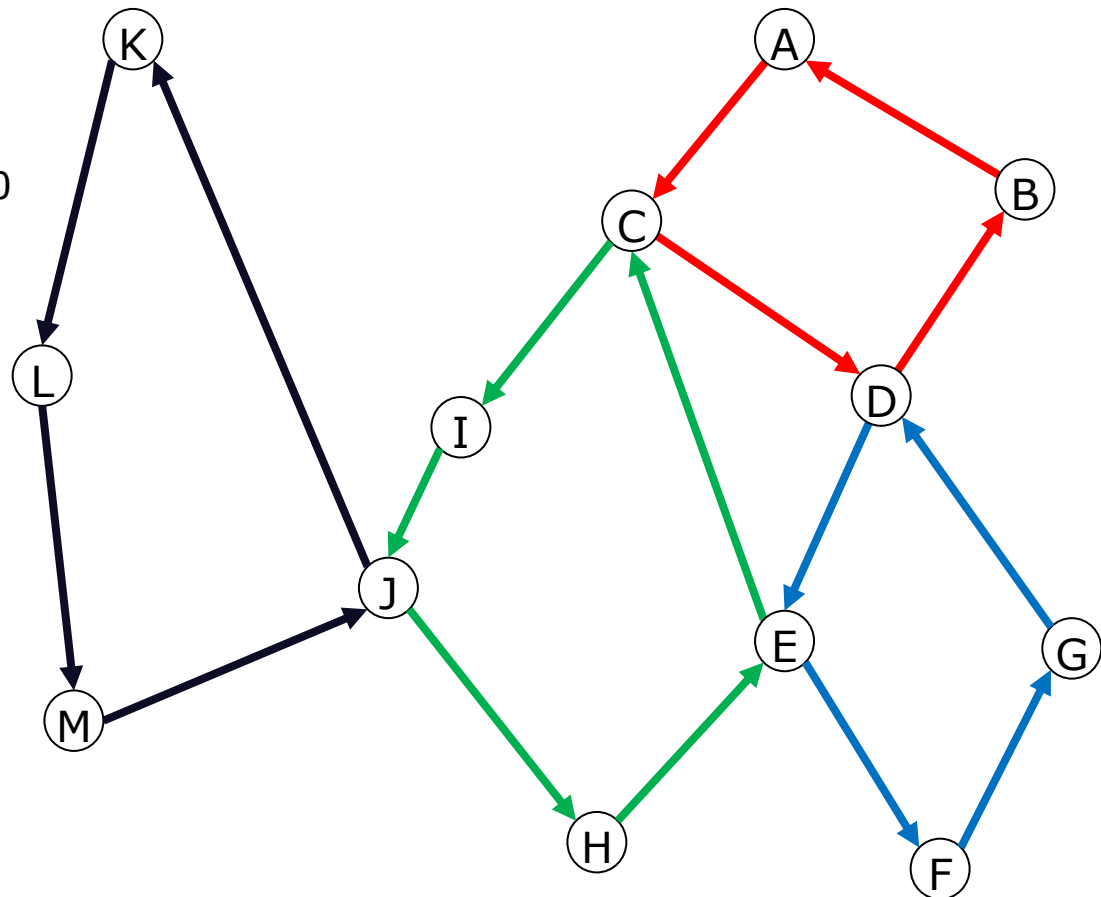
DTU "abstract" network

- Expected transport demand drives the design of the network. I.e.
 - 100 containers from A to M per week
 - 200 containers from L to G per week
 - 400 containers from C to E per week



1st problem: routing of containers

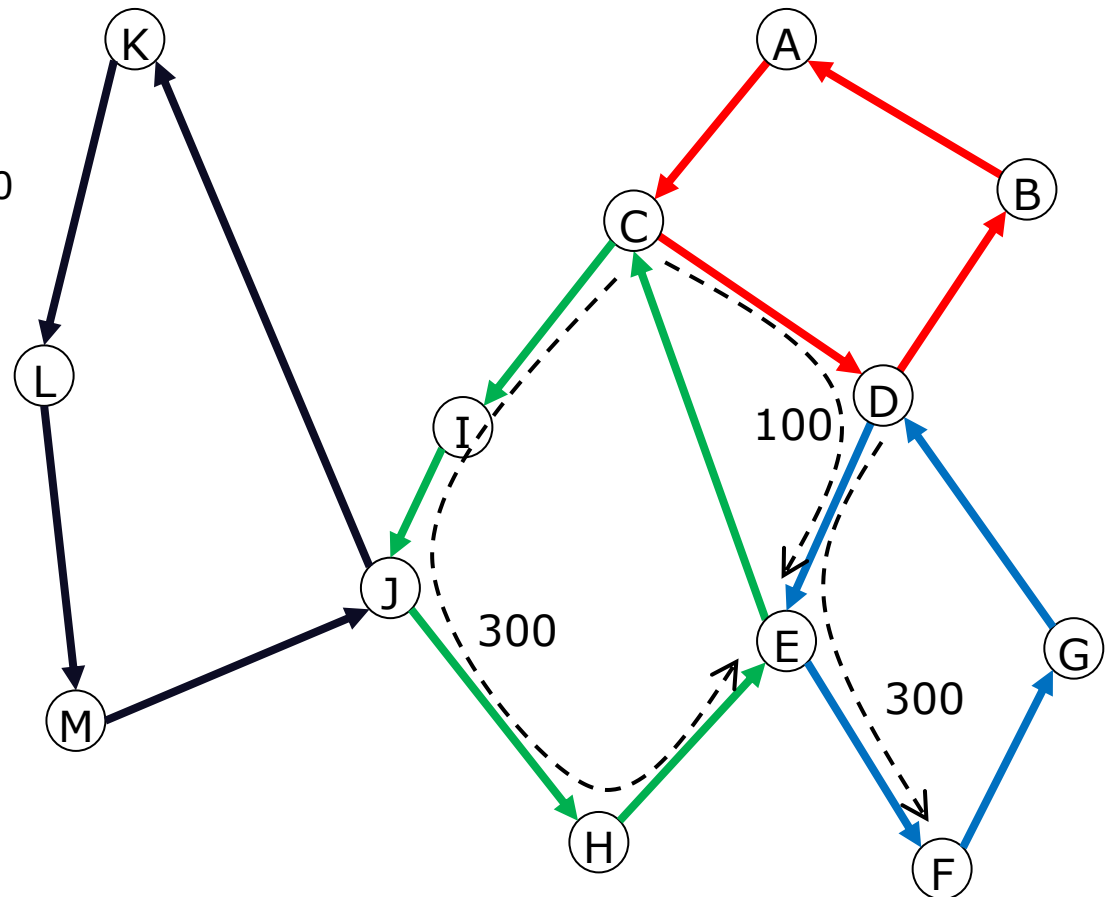
- Given network and transport demand. How should containers be routed?
- Capacity on each arc (depending on ships sailing on the rotation)
- 400 containers from C to E
- 300 containers from D to F
- Capacity of green rotation = 500
- Capacity of light blue rotation 400



1st problem: routing of containers

- Given network and transport demand. How should containers be routed?
- Capacity on each arc (depending on ships sailing on the rotation)

- 400 containers from C to E
- 300 containers from D to F
- Capacity of green rotation = 500
- Capacity of light blue rotation = 400



1st problem: routing of containers

Parameters and sets	
N	Number of nodes
A	Arcs
K	Number of commodities
d^k	Amount of commodity k available
$s(k)$	Source vertex for commodity k
$e(k)$	Destination vertex for commodity k
c_{ijk}	Cost of sending one unit of commodity k along arc (i, j)
u_{ij}	Capacity of arc (i, j)
Decision variables	
x_{ijk}	Amount of commodity k flowing on arc (i, j)

$$d(i, k) = \begin{cases} d^k & \text{if } i = s(k) \\ -d^k & \text{if } i = e(k) \\ 0 & \text{otherwise} \end{cases}$$

$$\min \sum_{(i,j) \in A} \sum_{k=1}^K c_{ijk} x_{ijk} \quad (1)$$

subject to

$$\sum_{j=1}^N x_{ijk} - \sum_{j=1}^N x_{jik} = d(i, k) \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (2)$$

$$\sum_{k=1}^K x_{ijk} \leq u_{ij} \quad \forall (i, j) \in A \quad (3)$$

$$x_{ijk} \geq 0 \quad \forall (i, j) \in A, k = 1, \dots, K \quad (4)$$

Multi commodity flow problem

$$\min \sum_{(i,j) \in A} \sum_{k=1}^K c_{ijk} x_{ijk} \quad (1)$$

subject to

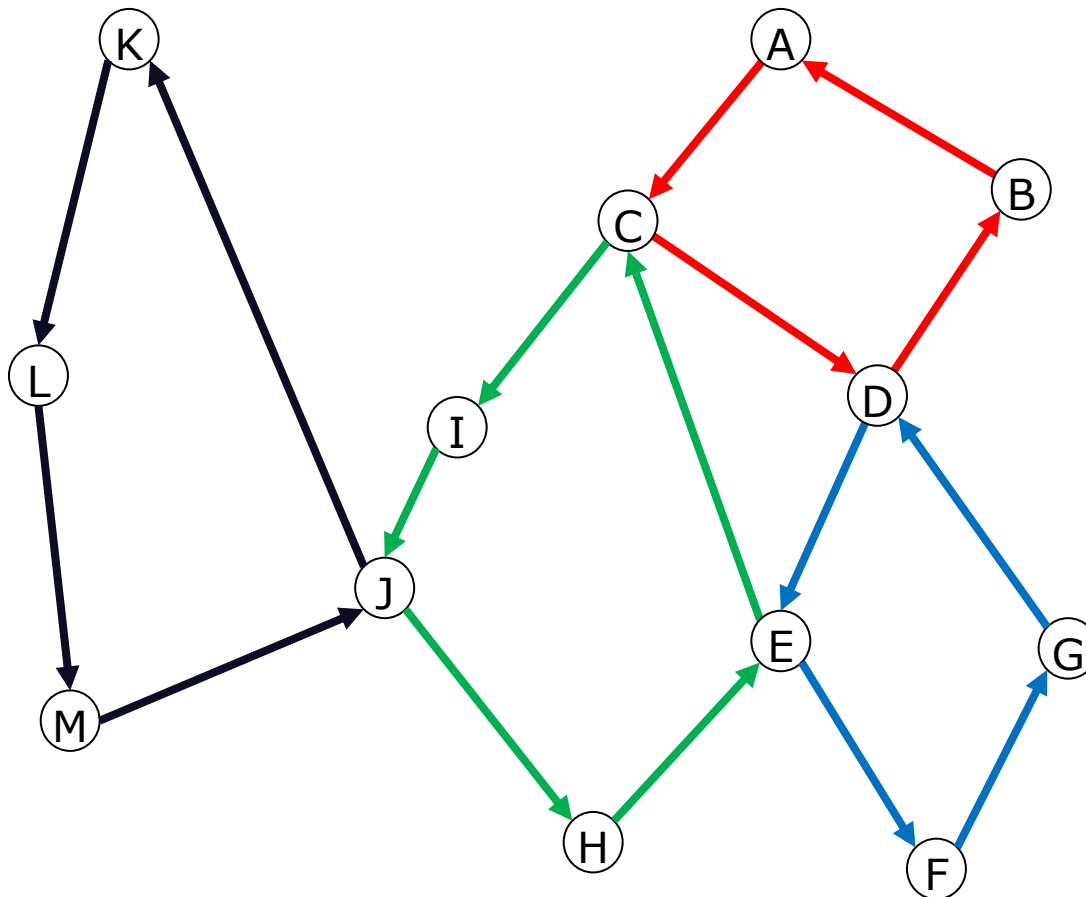
$$\sum_{j=1}^N x_{ijk} - \sum_{j=1}^N x_{jik} = d(i, k) \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (2)$$

$$\sum_{k=1}^K x_{ijk} \leq u_{ij} \quad \forall (i, j) \in A \quad (3)$$

$$x_{ijk} \geq 0 \quad \forall (i, j) \in A, k = 1, \dots, K \quad (4)$$

- Is it ok that x_{ijk} can be fractional?
- Is it ok that containers with same source and destination take different routes?
- What if we do not want to (or cannot) ship all commodities?

- We know how to route the containers
- But how do we design the network?



DTU Multi commodity fixed charge network design problem

Parameters and sets	
N	Number of nodes
A	Arcs
K	Number of commodities
d^k	Amount of commodity k available
$s(k)$	Source vertex for commodity k
$e(k)$	Destination vertex for commodity k
c_{ijk}	Cost of sending one unit of commodity k along arc (i, j)
f_{ij}	Cost of opening / activating arc a
u_{ij}	Capacity of arc (i, j)
Decision variables	
$x_{ijk} \geq 0$	Amount of commodity k flowing on arc (i, j)
$y_{ij} \in \{0, 1\}$	Arc (i, j) open (1) or closed (0).

$$d(i, k) = \begin{cases} d^k & \text{if } i = s(k) \\ -d^k & \text{if } i = e(k) \\ 0 & \text{otherwise} \end{cases}$$

DTU Multi commodity fixed charge network design problem

$$\min \sum_{(i,j) \in A} \sum_{k=1}^K c_{ijk} x_{ijk} + \sum_{(i,j) \in A} f_{ij} y_{ij} \quad (1)$$

subject to

$$\sum_{j=1}^N x_{ijk} - \sum_{j=1}^N x_{jik} = d(i, k) \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (2)$$

$$\sum_{k=1}^K x_{ijk} \leq u_{ij} y_{ij} \quad \forall (i, j) \in A \quad (3)$$

$$x_{ijk} \geq 0 \quad \forall (i, j) \in A, k = 1, \dots, K \quad (4)$$

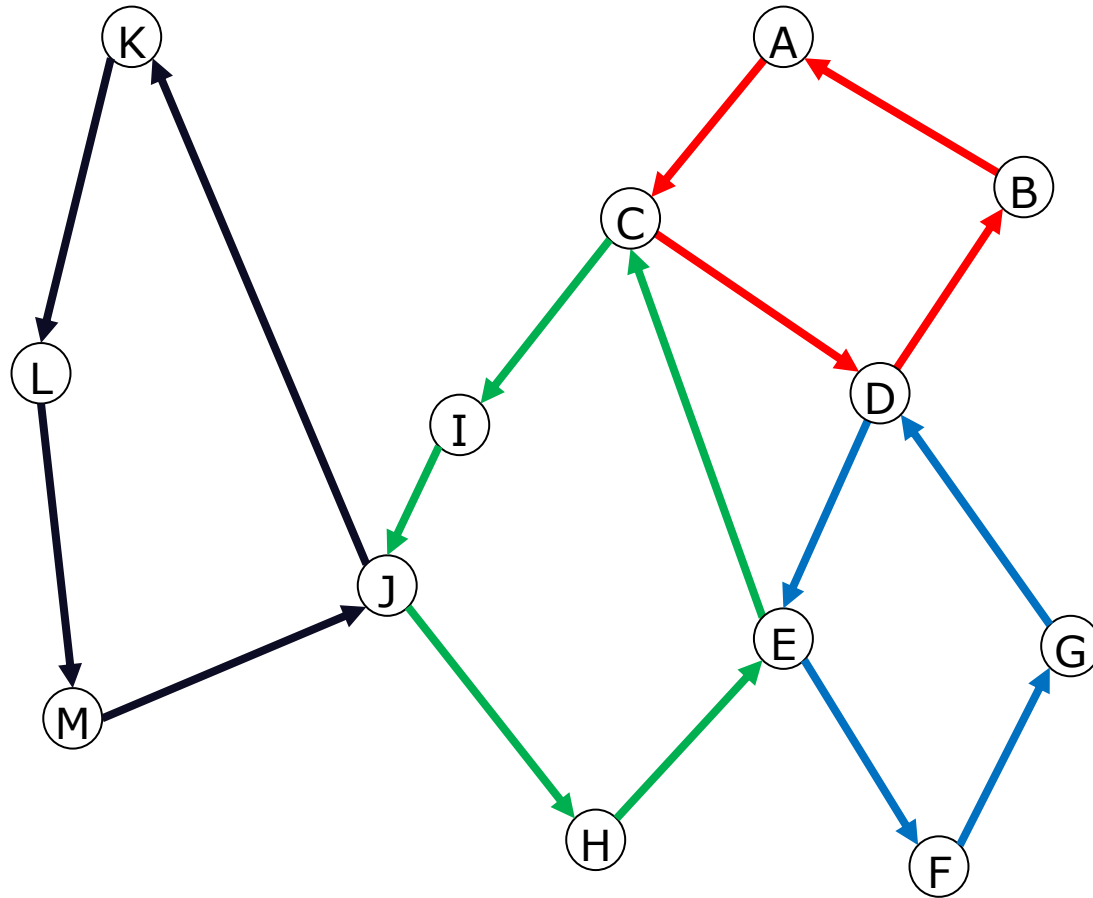
$$y_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (5)$$

DTU Multi commodity fixed charge network design problem

- Model is rather difficult to solve and sensitive to number of commodities
- 20 vertices, 228 arcs, 39 commodities
 - Solved to optimality within a few seconds (using CPLEX)
- 20 vertices, 229 arcs, 200 commodities
 - Not solved within an hour (using CPLEX)

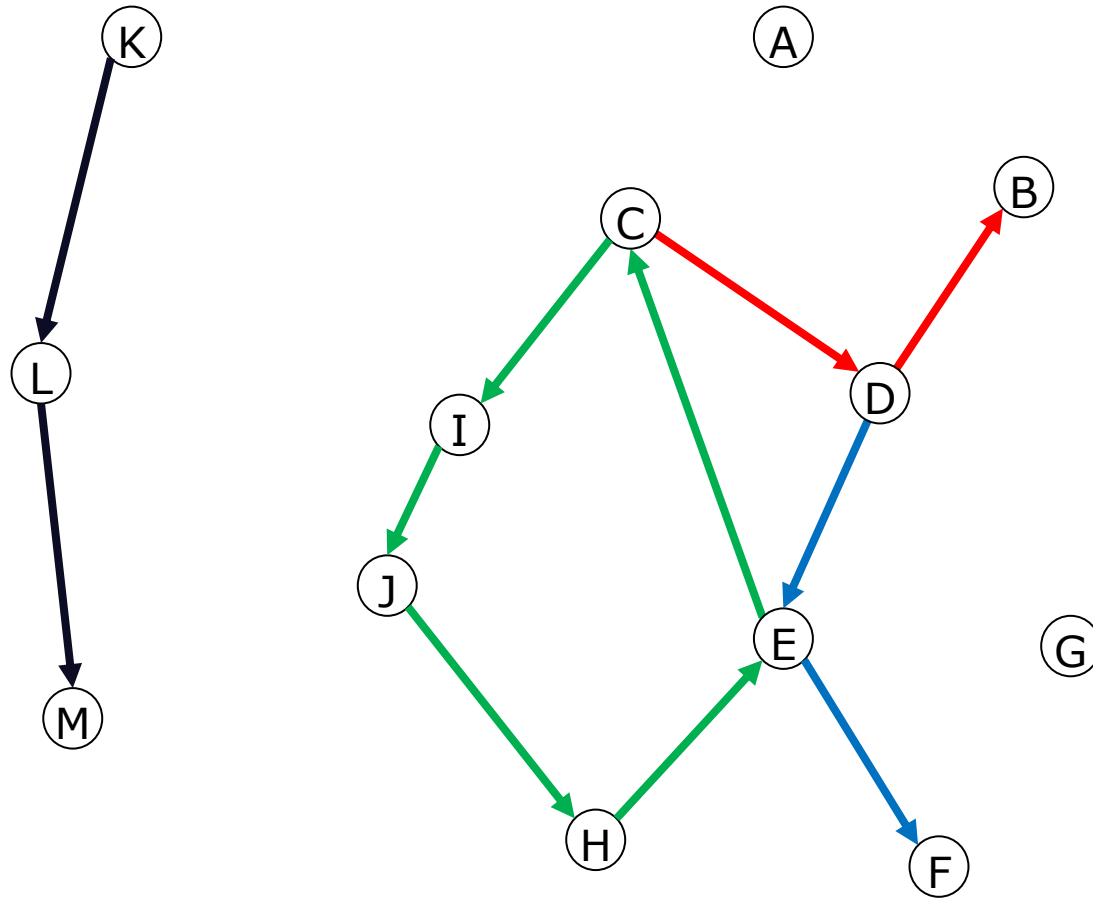
DTU Multi commodity fixed charge network design problem

We wanted something like this, but....



DTU Multi commodity fixed charge network design problem

Multi commodity fixed charge network design problem generates something like:



42112 – Lecture 6

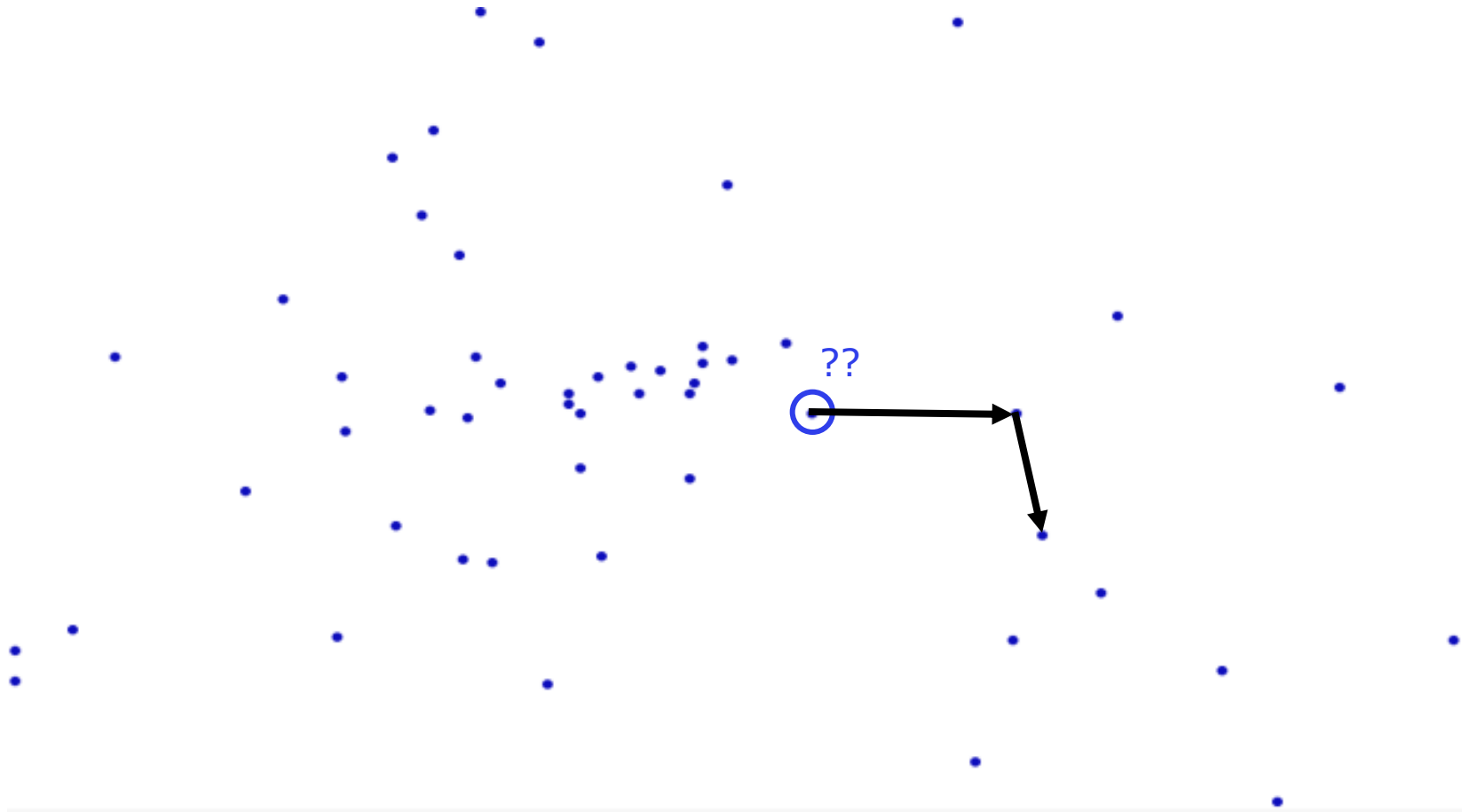
Vehicle routing problems



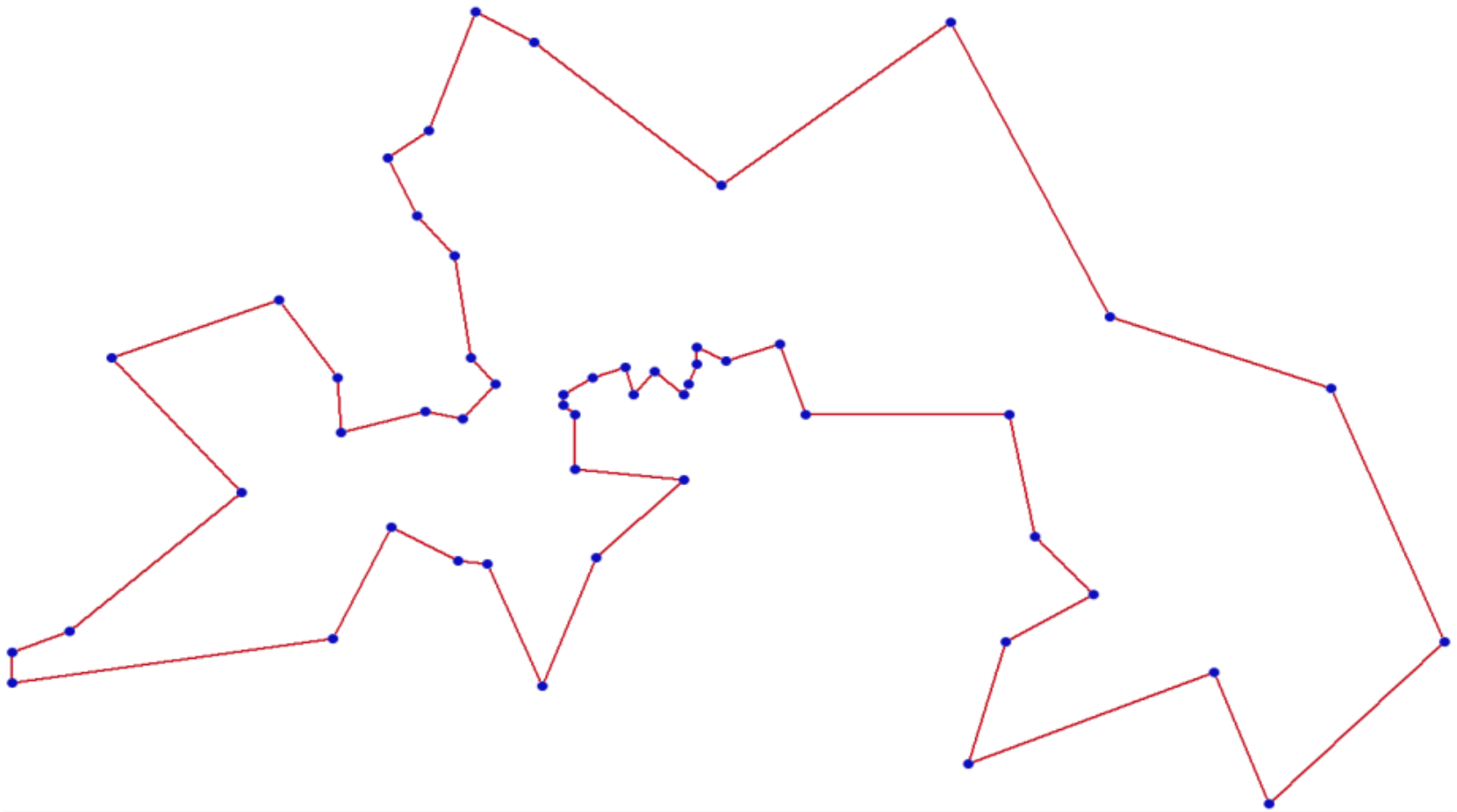
Agenda

- Traveling salesman problem
- Vehicle routing problem

The traveling salesman problem (TSP)



The traveling salesman problem



The traveling salesman problem

- The TSP is not famous because of the applications but because:
 - It's easy to understand, but difficult to solve.
 - Has been at the front of OR research since the beginning.
 - Many discoveries have started with the TSP.
 - Excellent problem for teaching/learning OR-techniques.
 - One of the most studied problem within operations research.

TSP model (Miller-Tucker-Zemlin)

Notation:

- N : number of nodes
- c_{ij} : cost of going from node i to node j

Variables

- $x_{ij} \in \{0, 1\}$: 1 if arc (i, j) is used, 0 otherwise
- u_i : counter, helps us eliminate sub-tours

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

subject to

$$\sum_{i=1}^N x_{ij} = 1 \quad \forall j = 1, \dots, N$$

$$\sum_{i=1}^N x_{ji} = 1 \quad \forall j = 1, \dots, N$$

$$u_i + 1 \leq u_j + N(1 - x_{ij}) \quad \forall i = 1, \dots, N, j = 2, \dots, N$$

$$\sum_{i=1}^N x_{ii} = 0$$

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, N, j = 1, \dots, N$$

$$u_i \geq 0 \quad \forall i = 1, \dots, N$$

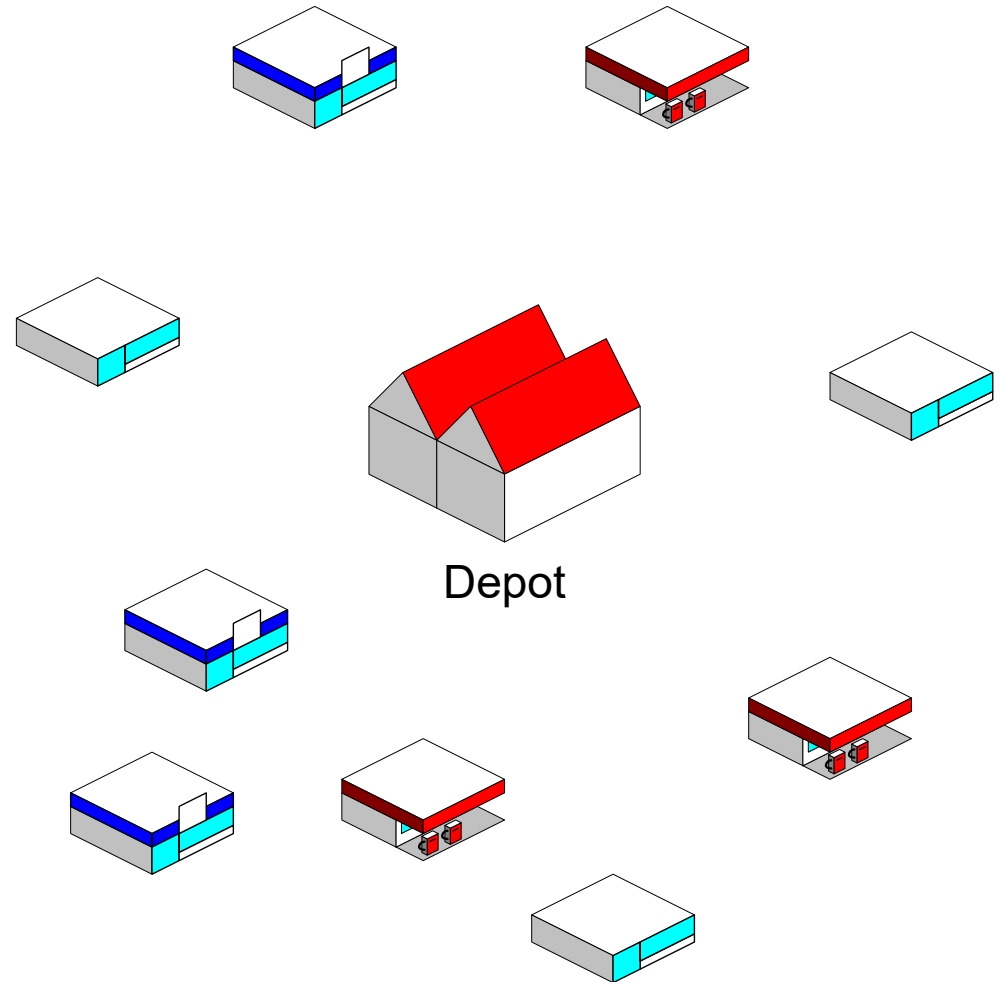


TSP Variants

- Precedence constrained TSP
- TSP with pickup and delivery
- TSP with time windows

DTU Vehicle routing problems

Our task: Supply supermarkets and gas stations with good from a common warehouse using three trucks

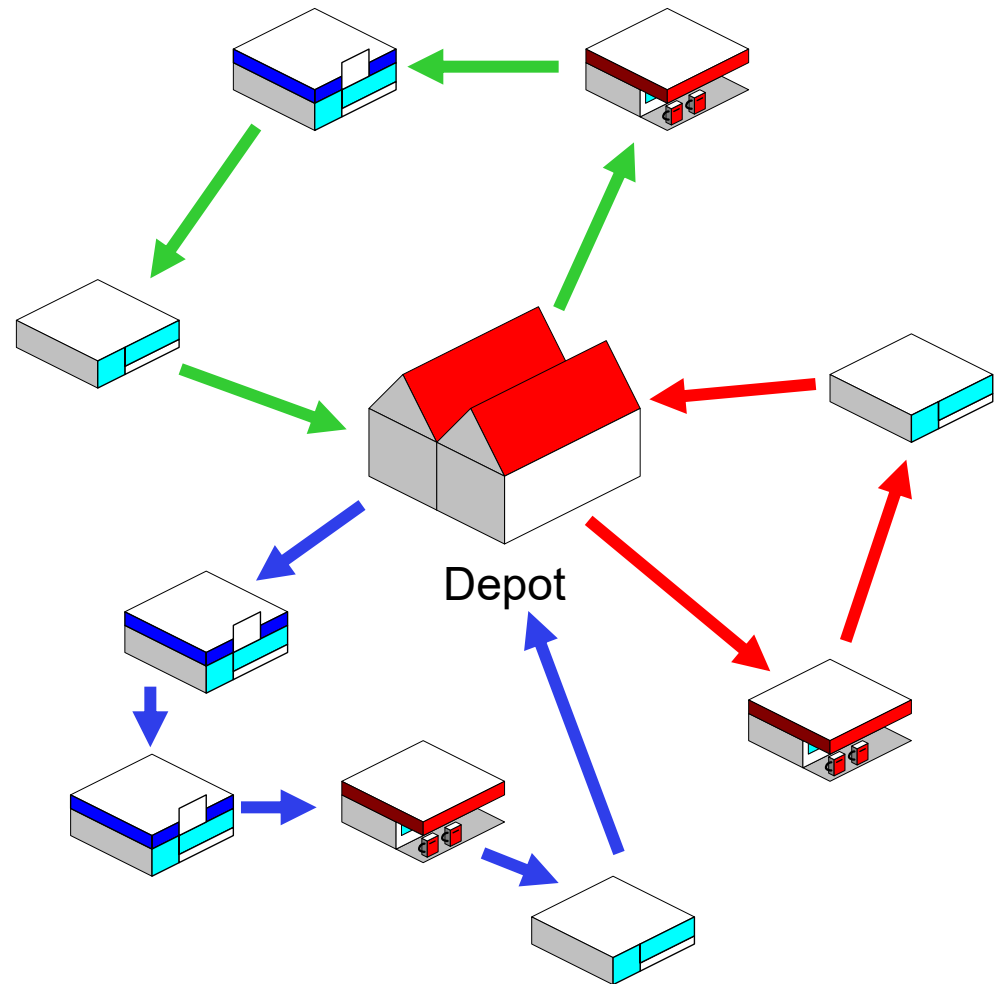


DTU Vehicle routing problems

Our task: Supply supermarkets and gas stations with good from a common warehouse using three trucks.

Terminology:

- Depot
- Customer (TSP: city)
- Route
- Vehicle

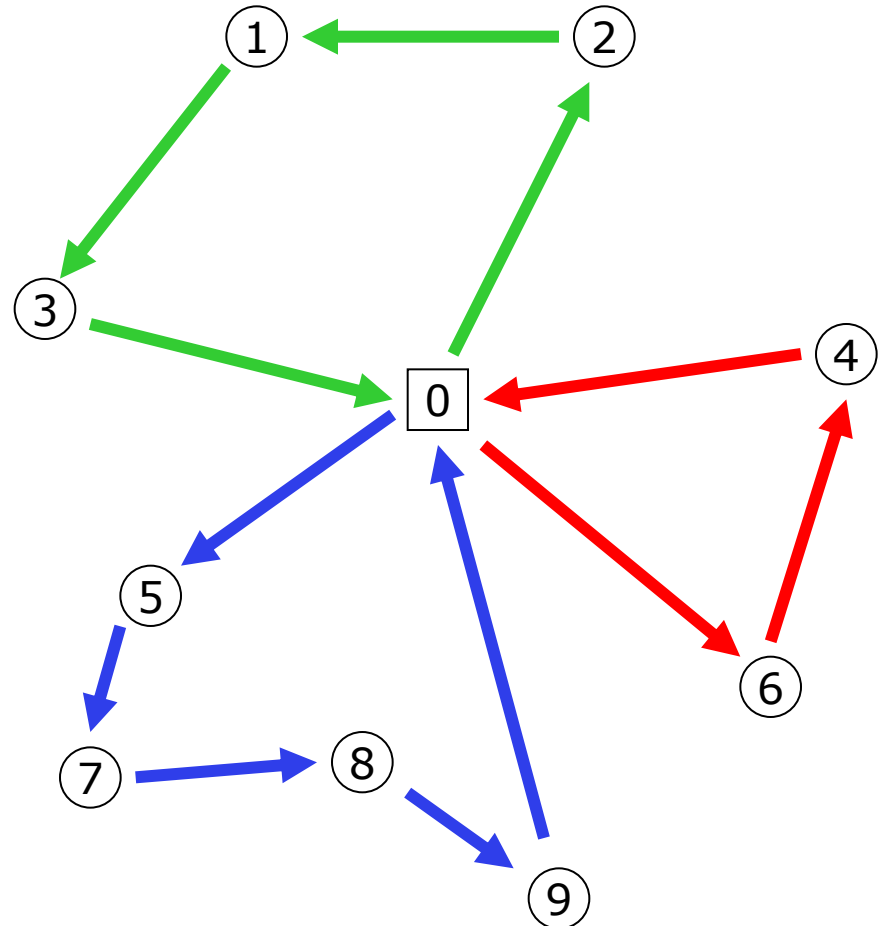


DTU Vehicle routing problems

Our task: Supply supermarkets and gas stations with good from a common warehouse using three trucks.

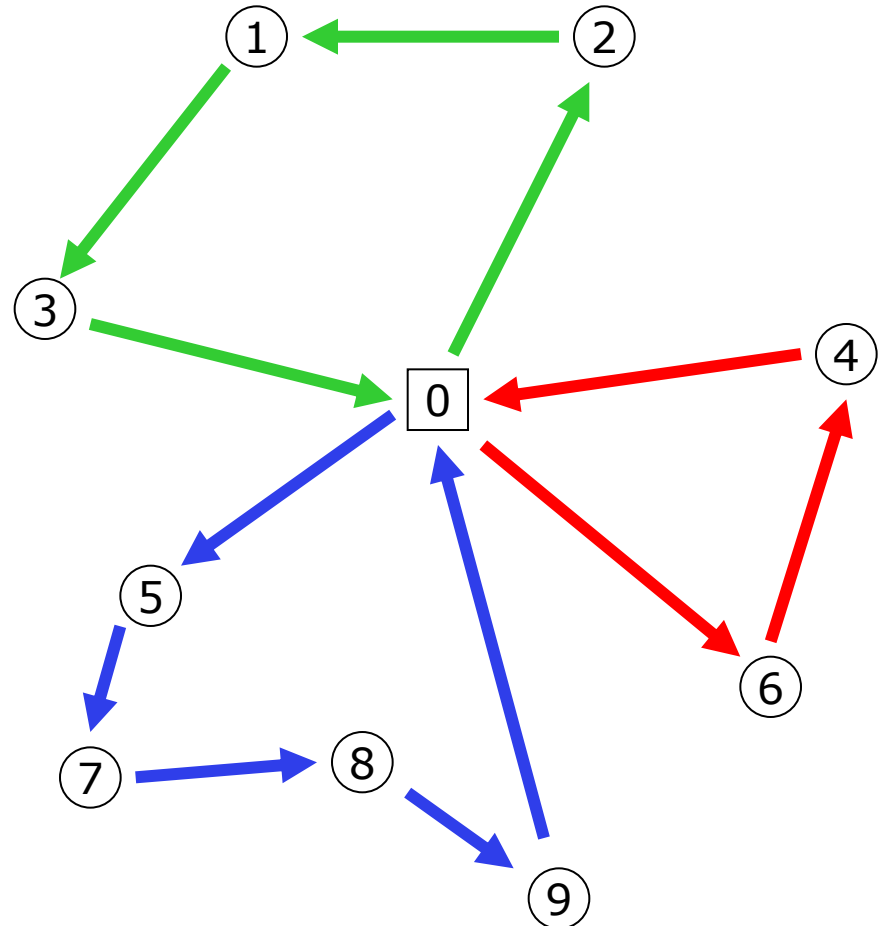
Terminology:

- Depot
- Customer (TSP: city)
- Route
- Vehicle



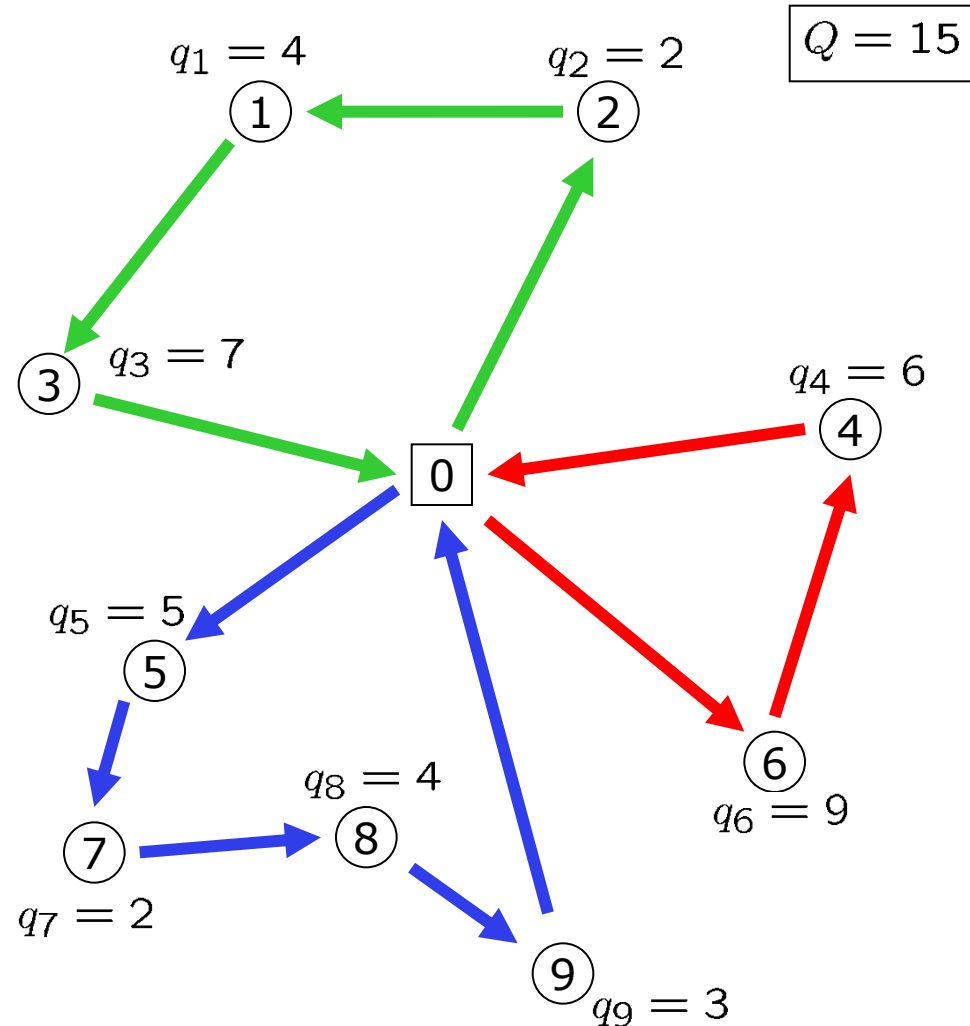
Capacitated vehicle routing problem (CVRP):

- Visit all customers once.
- Each customer has a demand q_i .
- Each vehicle has a capacity Q .
- Sum of demands on a route $\leq$ vehicle capacity
- Number of routes should be equal to K (variant 1).
- **Or:** Number of routes should be less than or equal to K (variant 2).
- **Objective:** Minimize cost of routes.



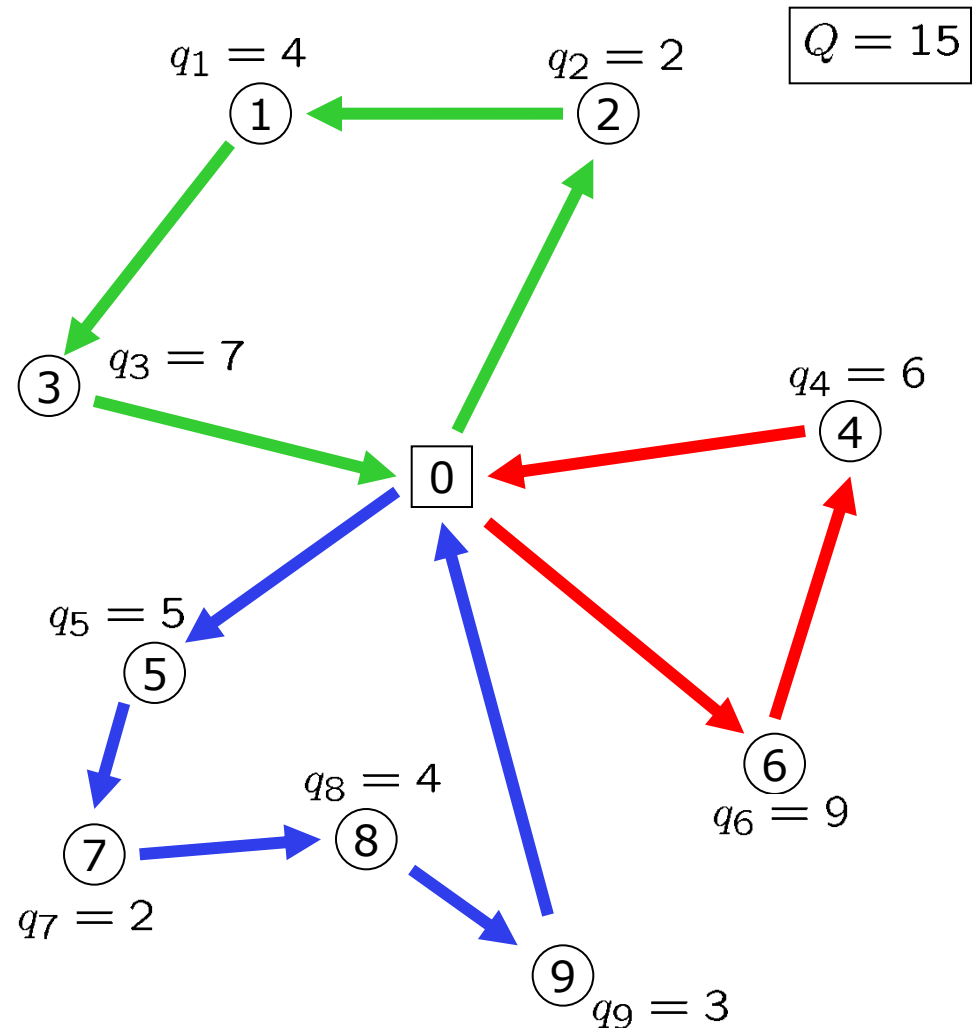
Capacitated vehicle routing problem (CVRP):

- Visit all customers once.
- Each customer has a demand q_i .
- Each vehicle has a capacity Q .
- Sum of demands on a route $\leq$ vehicle capacity
- Number of routes should be equal to K (variant 1).
- **Or:** Number of routes should be less than or equal to K (variant 2).
- **Objective:** Minimize cost of routes.



Capacitated vehicle routing problem (CVRP):

- Much harder to solve than the TSP.
- Largest (nontrivial) instance solved to optimality:
 - CVRP: 856 customers.
 - TSP: 85900 cities.





Vehicle routing problems

- Vehicle routing problems occur frequently in practice.
- Large savings can be realized by using computerized optimization of the distribution planning. 5-20%. (?)
- Some companies specializes in making vehicle routing problem software - several in Denmark.
- Many companies in Denmark use some kind of software to control their fleet of vehicles.
- Many variants of the VRP motivated from real life applications.

Vehicle routing problems - model

Notation:

- N : number of nodes. Node 1 is the depot, $2, \dots, N$ are customers
- c_{ij} : cost of going from node i to node j
- q_i : demand of customer i
- Q : vehicle capacity

Variables

- $x_{ij} \in \{0, 1\}$: 1 if arc (i, j) is used, 0 otherwise
- u_i : counter, keeps track of vehicle load

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

$$\begin{aligned} \text{s.t. } & \sum_{i=1}^N x_{ij} = 1 && \forall j = 1, \dots, N \\ & \sum_{i=1}^N x_{ji} = 1 && \forall j = 1, \dots, N \\ & u_i + q_j \leq u_j + Q(1 - x_{ij}) && \forall i = 1, \dots, N, j = 2, \dots, N \\ & \sum_{i=1}^N x_{ii} = 0 \\ & x_{ij} \in \{0, 1\} && \forall i = 1, \dots, N, j = 1, \dots, N \\ & 0 \leq u_i \leq Q && \forall i = 1, \dots, N \end{aligned}$$

VRP Variants

- VRP with time windows
- Pickup and delivery problems
- Multi-depot problems
- 2-echelon problems
- VRP with drones

DTU



Math-Heuristics: A Brief Introduction

Lecture 9

Learning Objectives

What will be presented in this lecture?

- Why math-heuristics?
- How to use MIP optimization as a component
- Programming structure (yes actual programming!)
- An example!
- Types of math-heuristics
- Debugging Advice (OR Models)
- Literature list

Why Use Math-Heuristics?

What to do, when your important MIP model (e.g. your BSc/MSc project model) cannot be solved ?



The Importance of Being Able to SOLVE the Models

If you want to solve real problems, making the model is not enough (we cannot sell mathematics to industry, we have to deliver solutions. Luckily for you, we have courses teaching you how to solve hard/large optimization problems:

- 42136: "Large Scale Optimization using Decomposition", Spring. This course apply advanced math to optimize the hard MIP models. Pre-requisites: 42114 "Integer Programming"
- 42137: "Optimization using Metaheuristics", Spring. This course teach you to program you own metaheuristic solvers. Pre-requisites: Programming

Ease of Implementation

Because we use Julia/JuMP, we can relatively easily build optimization programs "around" our model. This opens up possibilities for you. But it **DOES** require a little more programming.

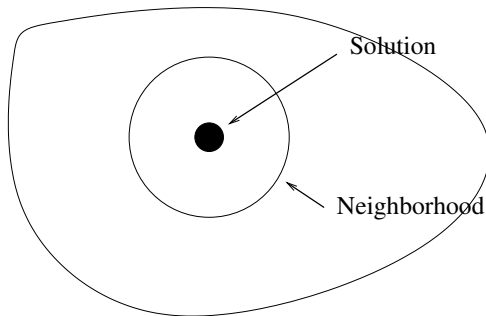
Programming Constructs Necessary

- If statements
- While statements
- Function statements
- Specialized JuMP functions:
 - Fix, is_fixed, unfix (fix-and-optimize heuristic)
 - delete, unregister (hamming-distance heuristic)

What Is a Heuristic?

- 'Intuition'-based method for solving a problem
- Typically not guaranteed to find the optimal solution
- We move in steps (neighbourhoods) in hope of finding a better solution
- Require initial solution: Random, 'zero'-solution, current solution, create a simple solution
- Two applications:
 - Construction: 'Greedy heuristics' used to construct an initial solution
 - Improvement: Improve the quality of the current solution. Math-heuristics are typically improving

What Is a Neighborhood ?



Given a solution, i.e. a seating plan a wedding table plan, the neighbourhood is a set of other seating plans which are "similar". This means that the binary variable values are partly shared.

The Two Simplest Algorithms

- Hamming Distance Neighbourhood: At most change K binary variables compared to the current solution. Given N binary variables, the number of possible solutions is at most: $\frac{N!}{(N-K)!}$. Controlled via a constraint
- Fix-and-optimize Neighbourhood: Only allow to change some of the binary variables. If you have K free binary variables this is equal to 2^K variables. Controlled via variable (un)fixing

Hamming Distance Algorithm

- Get a feasible solution, this is the initial solution
- Add Hamming-distance constraint: '*At most change K variables*'
- Optimize
- Repeat

The Hamming Distance Algorithm

The Hamming distance approach is rather simple if the variables are just binary variables. Assume the following example. Given a set of binary variables $x_i \in \{0, 1\}$, a current solution $\bar{x}$ and a limit on how many binary variables can change. We simply add:

$$\sum_{i|\bar{x}_i=0} x_i + \sum_{i|\bar{x}_i=1} (1 - x_i) \leq K$$

The Hamming Distance Algorithm (Pseudo-Code)

```
1 Set curSol,curObj,oldObj,K;  
2 while time < limit do  
3   (newObj,newSol,newSlack)=AddKConstraintAndOptimize(curSol,K);  
4   curSol=newSol;  
5   if newObj ≤ curObj then  
6     | K=K+2;  
7   if newObj > curObj then  
8     | K=2;
```

Hamming Distance Algorithm (Julia)

```
while (time_ns()-start_time)/1.0e9<time_limit
    (curObj,xVal,slackVal)=AddKConstraintAndOptimize(xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(this_time) K: $(K) CurObj: $(curObj)")
    if curObj<=oldObj # comparison for maximization problems
        K=K+1
    else
        K=2
    end
    oldObj=curObj
    it+=1
end
```

Adding a Hamming Distance in Julia/JuMP

```
function AddKConstraintAndOptimize(xVal,K)
@constraint(wp, Kconstraint,
  sum( x[g,t] for g=1:G,t=1:T if xVal[g,t]==0) +
  sum( (1-x[g,t]) for g=1:G,t=1:T if xVal[g,t]==1)<= K)
optimize!(wp)
xVal=round.(Int,value.(x))
slackVal=round.(Int,value.(slack))
cur_obj=round(Int64,objective_value(wp))
delete(wp, Kconstraint)
unregister(wp, :Kconstraint)
return (cur_obj,xVal,slackVal)
end
```

Fix-and-Optimize Version

Instead of allowing all (binary) variables to change and bound it by a constraint, we will only consider a limited set of variables. This small set of variables are all allowed to change, however, the majority of decision variables are fixed.

Fix-and-Optimize Algorithm

- Get a feasible solution, this is the initial solution
- Fix all variables to the current solution, and then free K random variables
- Optimize
- Repeat

Fix-and-Optimize Algorithm (Pseudo-Code)

```
1 Set curSol,curObj,oldObj,K;  
2 while time < limit do  
3   (newObj,newSol,newSlack)=FixAndOptimize(curSol,K);  
4   if newObj  $\geq$  curObj then  
5     curSol=newSol;
```

Fixing the Variables To a Value in Julia/JuMP

First, fix all variables to current value:

```
function FixAndOptimize(xVal,K)
    # first lock all values
    for g=1:G
        for t=1:T
            fix(x[g,t], xVal[g,t]; force = true)
        end
    end
    ...
end
```


Freeing a Subset of Variables in Julia/JuMP

Now free K variables:

```
function FixAndOptimize(xVal,K)
    ...
    # then free K
    free_vars=0
    while free_vars<K
        g=rand(1:G)
        t=rand(1:T)
        if is_fixed(x[g,t])
            unfix(x[g,t])
            free_vars+=1
        end
    end
end
```

Solve After Fix-Unfix in Julia/JuMP

Finally, optimize and return the result:

```
function FixAndOptimize(xVal,K)
    ...
    optimize!(wp)
    xVal=round.(Int,value.(x))
    slackVal=round.(Int,value.(slack))
    cur_obj=round(Int64,objective_value(wp))
    return (cur_obj,xVal,slackVal)
end
```

Overall Heuristic Fix-and-Optimize Heuristic

```
while this_time<time_limit
    (cur_obj,cur_xVal,slackVal)=FixAndOptimize(cur_xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(this_time) K: $(K) CurObj: $(cur_obj)")

    if cur_obj>old_obj+0.5 # comparison for maximization problems
        old_obj=cur_obj
    end

    it+=1
end
```

What is the Value of K in Fix-and-Optimize ?

With a time limit of 120 seconds, I tried different K values:

- Iterations: 8009 Objective: 285 K: 100
- Iterations: 4258 Objective: 307 K: 200
- Iterations: 3359 Objective: 315 K: 300
- Iterations: 2474 Objective: 329 K: 400
- Iterations: 269 Objective: 331 K: 500
- Iterations: 3 Objective: 328 K: 600

Example: WeddingPlanner

We will use the wedding planner as an example

- Model update: Maximize the shared interests, but **allow** un-seated guests
- Initial solution: No seated guests
- Hamming-distance version, allow K guest changes

Wedding Plan Model Changes: Allow Un-Seated Guests

```
@variable(wp, slack[g=1:G], Bin)
@constraint(wp, [g=1:G], slack[g] + sum( x[g,t] for t=1:T) == 1)
```

Wedding Plan Model: Hamming-Distance Constraint

```
sum( x[g,t] for g=1:G,t=1:T if xVal[g,t]==0) +  
sum( (1-x[g,t]) for g=1:G,t=1:T if xVal[g,t]==1)<= K)
```

Different Math-heuristics

There are a number of different math-heuristics suggestions, but actually not that many:

- Local Branching, [Fischetti and Lodi, 2003]
- Cut & Solve, [Climer and Zhang, 2006]
- Proximity search, [Fischetti and Monaci, 2014]

These heuristics are **NOT** part of this course

Few Tips on Debugging OR Models (in Julia)

You have made a beautiful mathematical model, but after entering it in Julia, it returns a wrong objective value or an infeasible solution... Try out these steps in order to identify (debug) the issue:

- Print solution values: Using the function `value(x[i,j])`, see the found solutions variables and verify. Maybe print a limited amount of them and/or try to visualize the solution (on paper). Note, this is not possible if the solution is infeasible

Few Tips on Debugging OR Models (in Julia)

You have made a beautiful mathematical model, but after entering it in Julia, it returns a wrong objective value or an infeasible solution... Try out these steps in order to identify (debug) the issue:

- **Print constraints:** Some typos or mistakes in the input data might have caused your entered model in Julia to look weird. Print the model to see how the constraints actually look like and verify if they match your mathematical model. If your model contains many constraints, print them one by one. Note that when printing the constraints, Julia will have moved all variables to the left-hand-side

Few Tips on Debugging OR Models (in Julia)

You have made a beautiful mathematical model, but after entering it in Julia, it returns a wrong objective value or an infeasible solution... Try out these steps in order to identify (debug) the issue:

- **Exclude Constraints:** Mostly relevant for when encountering an infeasible solution. Try excluding one constraint line at a time. This **might** help you identify the constraint that is causing the infeasibility. Try excluding all constraints to find all potential candidates. Note, this is just a tip and not guaranteed to find the cause, since the issue might happen across multiple constraint types!

Exercises Today

- Experiment with Hamming-distance and fix-and-optimize for Wedding planning: Download Programs from DTU-Learn and try them out.
- Solve the Stamp problem, now with many more stamps.

The size of the Stamp problem is tailored to be hard for the HiGHS solver. If you use Gurobi, you may be able to solve it to optimality.

Math-heuristics articles



Climer, S. and Zhang, W. (2006).

Cut-and-solve: An iterative search strategy for combinatorial optimization problems.

Artificial Intelligence, 170(8-9):714–738.



Fischetti, M. and Lodi, A. (2003).

Local branching.

Mathematical programming, 98:23–47.



Fischetti, M. and Monaci, M. (2014).

Proximity search for 0-1 mixed-integer convex programming.

Journal of Heuristics, 20:709–731.